What kind of code structure could allow this disastrous bug to slip through? We could only guess. We never received a full explanation.

What are some of the most impactful software bugs you encountered in your career?

AWS Lambda behind the scenes

Serverless is one of the hottest topics in cloud services. How does AWS Lambda work behind the scenes?

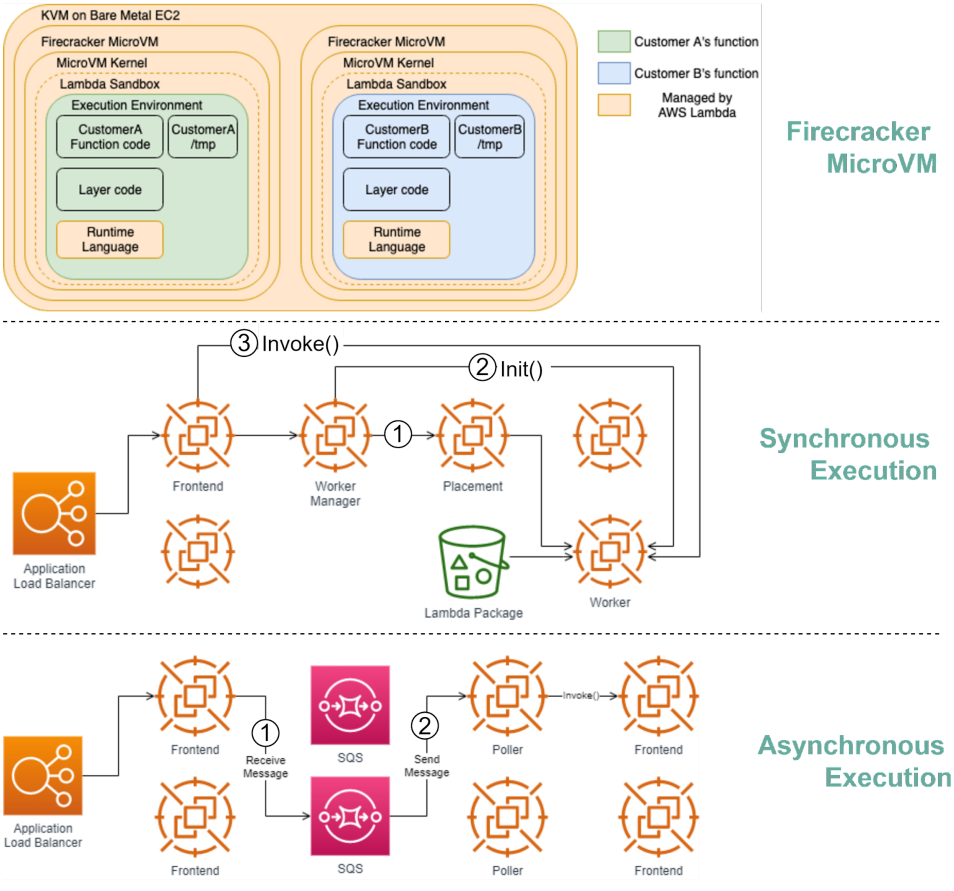
Lambda is a **serverless** computing service provided by Amazon Web Services (AWS), which runs functions in response to events.

Firecracker MicroVM

Firecracker is the engine powering all of the Lambda functions [1]. It is a virtualization technology developed at Amazon and written in Rust.

The diagram below illustrates the isolation model for AWS Lambda Workers.

How does AWS Lambda work?



Lambda functions run within a sandbox, which provides a minimal Linux userland, some common libraries and utilities. It creates the Execution environment (worker) on EC2 instances.

How are lambdas initiated and invoked? There are two ways.

Synchronous execution
Step1: "The Worker Manager communicates with a Placement Service which is responsible to place a workload on a location for the given host (it's provisioning the sandbox) and returns that to the Worker Manager" [2].

Step 2: "The Worker Manager can then call *Init* to initialize the function for execution by downloading the Lambda package from S3 and setting up the Lambda runtime" [2]

Step 3: The Frontend Worker is now able to call *Invoke* [2].

Asynchronous execution
Step 1: The Application Load Balancer forwards the invocation to an available Frontend which places the event onto an internal queue(SQS).
Step 2: There is "a set of pollers assigned to this internal queue which are responsible for polling it and moving the event onto a Frontend synchronously. After it's been placed onto the Frontend it follows the synchronous invocation call pattern which we covered earlier" [2].

Question: Can you think of any use cases for AWS Lambda?

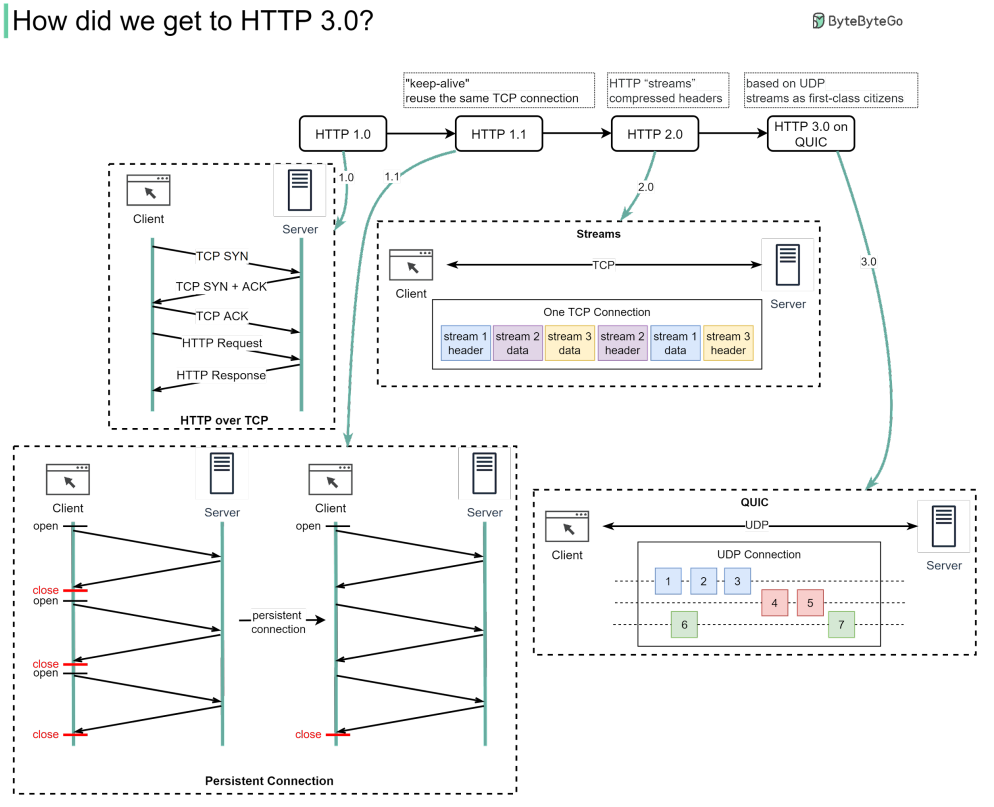
Sources:
[1] AWS Lambda whitepaper:
https://docs.aws.amazon.com/whitepapers/latest/security-overview-aws-lambda/lambda-executions.html
[2] Behind the scenes, Lambda:
https://www.bschaatsbergen.com/behind-the-scenes-lambda/

Image source: [1] [2]

HTTP 1.0 -> HTTP 1.1 -> HTTP 2.0 -> HTTP 3.0 (QUIC).

What problem does each generation of HTTP solve?

The diagram below illustrates the key features.



🔹 HTTP 1.0 was finalized and fully documented in 1996. Every request to the same server requires a separate TCP connection.

🔹 HTTP 1.1 was published in 1997. A TCP connection can be left open for reuse (persistent connection), but it doesn't solve the HOL (head-of-line) blocking issue.

HOL blocking - when the number of allowed parallel requests in the browser is used up, subsequent requests need to wait for the former ones to complete.

🔹 HTTP 2.0 was published in 2015. It addresses HOL issue through request multiplexing, which eliminates HOL blocking at the application layer, but HOL still exists at the transport (TCP) layer.

As you can see in the diagram, HTTP 2.0 introduced the concept of HTTP “streams”: an abstraction that allows multiplexing different HTTP exchanges onto the same TCP connection. Each stream doesn’t need to be sent in order.

🔹 HTTP 3.0 first draft was published in 2020. It is the proposed successor to HTTP 2.0. It uses QUIC instead of TCP for the underlying transport protocol, thus removing HOL blocking in the transport layer.

QUIC is based on UDP. It introduces streams as first-class citizens at the transport layer. QUIC streams share the same QUIC connection, so no additional handshakes and slow starts are required to create new ones, but QUIC streams are delivered independently such that in most cases packet loss affecting one stream doesn't affect others.

Question: When shall we upgrade to HTTP 3.0? Any pros & cons you can think of?

—
Check out our bestselling system design books.
Paperback: Amazon Digital: ByteByteGo.

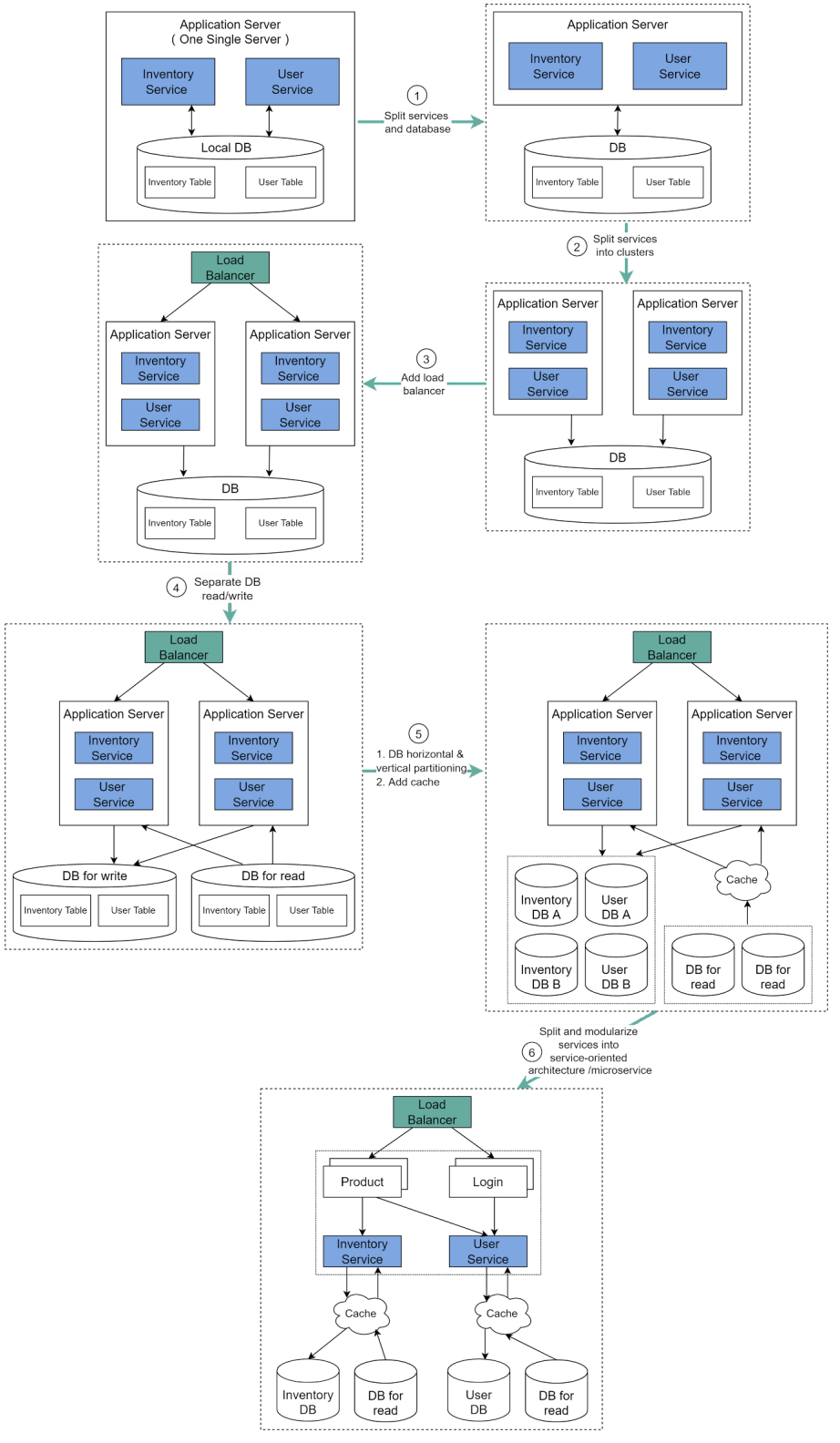
How to scale a website to support millions of users?

We will explain this step-by-step.

The diagram below illustrates the evolution of a simplified eCommerce website. It goes from a monolithic design on one single server, to a service-oriented/microservice architecture.

How to Scale a Website Step-by-Step?

ByteByteGo



Suppose we have two services: inventory service (handles product descriptions and inventory management) and user service (handles user information, registration, login, etc.).

Step 1 - With the growth of the user base, one single application server cannot handle the traffic anymore. We put the application server and the database server into two separate servers.

Step 2 - The business continues to grow, and a single application server is no longer enough. So we deploy a cluster of application servers.

Step 3 - Now the incoming requests have to be routed to multiple application servers, how can we ensure each application server gets an even load? The load balancer handles this nicely.

Step 4 - With the business continuing to grow, the database might become the bottleneck. To mitigate this, we separate reads and writes in a way that frequent read queries go to read replicas. With this setup, the throughput for the database writes can be greatly increased.

Step 5 - Suppose the business continues to grow. One single database cannot handle the load on both the inventory table and user table. We have a few options:

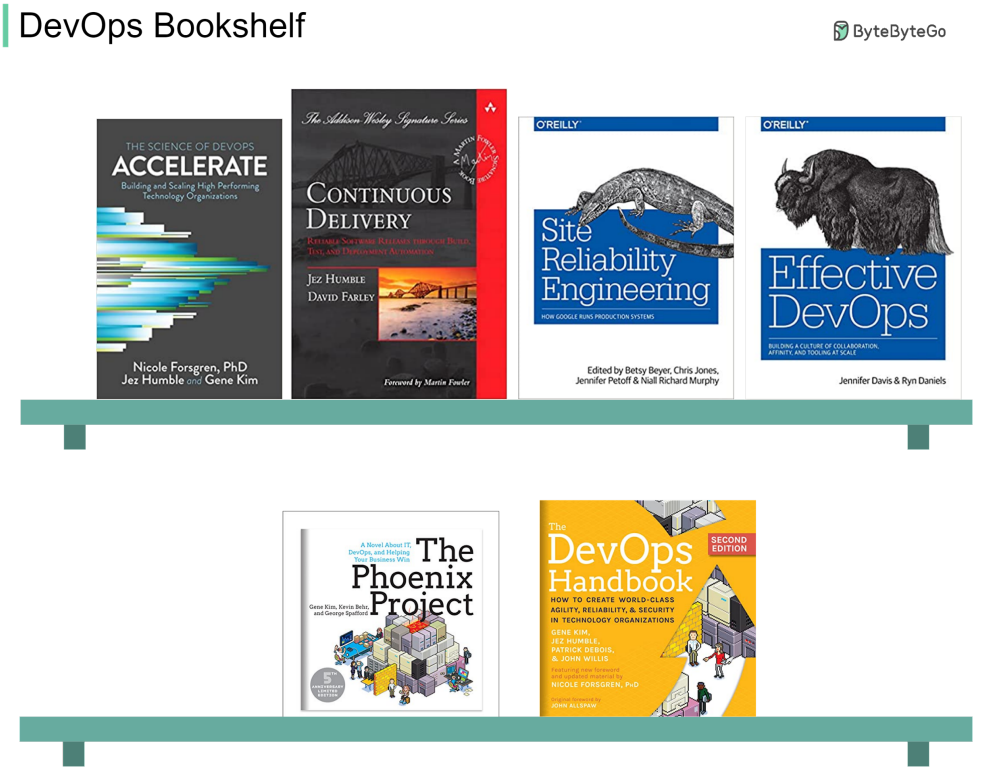
1. Vertical partition. Adding more power (CPU, RAM, etc.) to the database server. It has a hard limit.
2. Horizontal partition by adding more database servers.
3. Adding a caching layer to offload read requests.

Step 6 - Now we can modularize the functions into different services. The architecture becomes service-oriented / microservice.

Question: what else do we need to support an e-commerce website at Amazon’s scale?

DevOps Books

Some **DevOps** books I find enlightening:



🔹 Accelerate - presents both the findings and the science behind measuring software delivery performance.

🔹 Continuous Delivery - introduces automated architecture management and data migration. It also pointed out key problems and optimal solutions in each area.

🔹 Site Reliability Engineering - famous Google SRE book. It explains the whole life cycle of Google's development, deployment, and monitoring, and how to manage the world's biggest software systems.

🔹 Effective DevOps - provides effective ways to improve team coordination.

🔹 The Phoenix Project - a classic novel about effectiveness and communications. IT work is like manufacturing plant work, and a system must be established to streamline the workflow. Very interesting read!

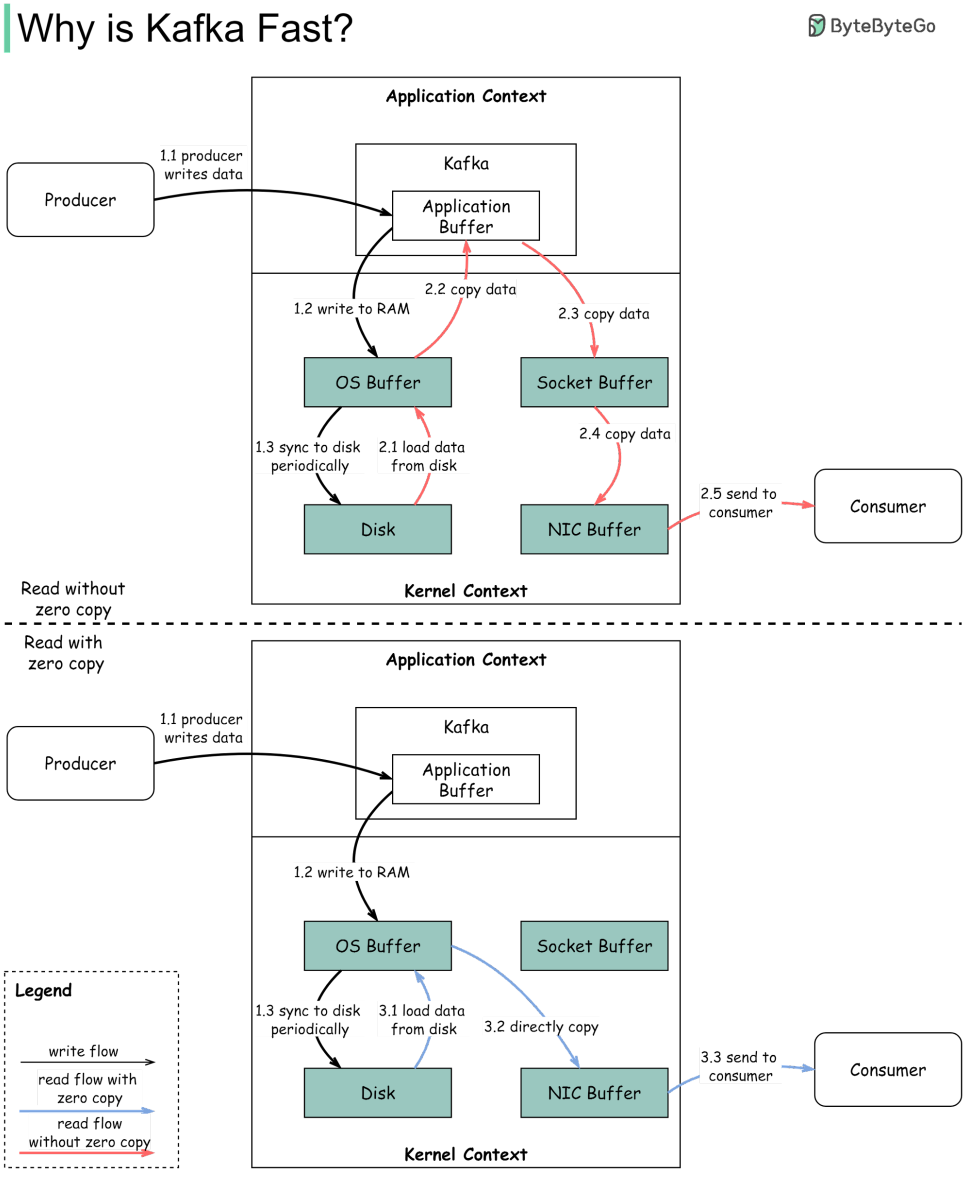
🔹 The DevOps Handbook - introduces product development, quality assurance, IT operations, and information security.

What’s your favorite dev-ops book?

Why is Kafka fast?

Kafka achieves low latency message delivery through Sequential I/O and Zero Copy Principle. The same techniques are commonly used in many other messaging/streaming platforms.

The diagram below illustrates how the data is transmitted between producer and consumer, and what zero-copy means.



- Step 1.1 - 1.3: Producer writes data to the disk

🔹 Step 2: Consumer reads data without zero-copy
2.1: The data is loaded from disk to OS cache
2.2 The data is copied from OS cache to Kafka application
2.3 Kafka application copies the data into the socket buffer
2.4 The data is copied from socket buffer to network card
2.5 The network card sends data out to the consumer

🔹 Step 3: Consumer reads data with zero-copy
3.1: The data is loaded from disk to OS cache
3.2 OS cache directly copies the data to the network card via sendfile() command
3.3 The network card sends data out to the consumer

Zero copy is a shortcut to save the multiple data copies between application context and kernel context. This approach brings down the time by approximately 65%.

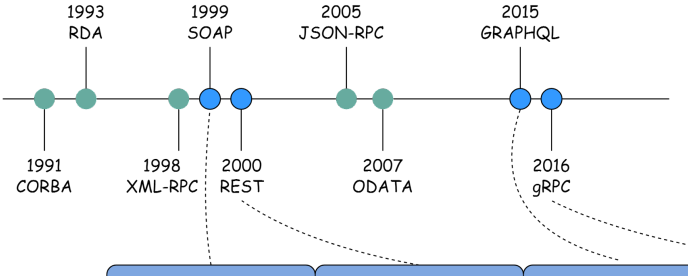
—

Check out our bestselling system design books.
Paperback: Amazon Digital: ByteByteGo.

SOAP vs REST vs GraphQL vs RPC.

The diagram below illustrates the API timeline and API styles comparison.

> API Architectural Styles Comparison



	SOAP (Simple Object Access Protocol)	REST (REpresentational State Transfer)	GraphQL	RPC (Remote Procedure Call)
Organized in terms of	enveloped message structure	compliance with six architectural constraints	schema & type system	local procedure call
Format	XML only	XML, JSON, HTML, plain text	JSON	JSON, XML, Protobuf, Thrift, FlatBuffers
Learning curve	Difficult	Easy	Medium	Easy
Community	Small	Large	Growing	Large
Use cases	- payment gateways - identity management - CRM solutions - financial and telecommunication services - legacy system support	- public APIs - simple resource-driven apps	- mobile APIs - complex systems - micro-services	- command and action-oriented APIs - high performance communication in massive micro-services systems

Over time, different API architectural styles are released. Each of them has its own patterns of standardizing data exchange.

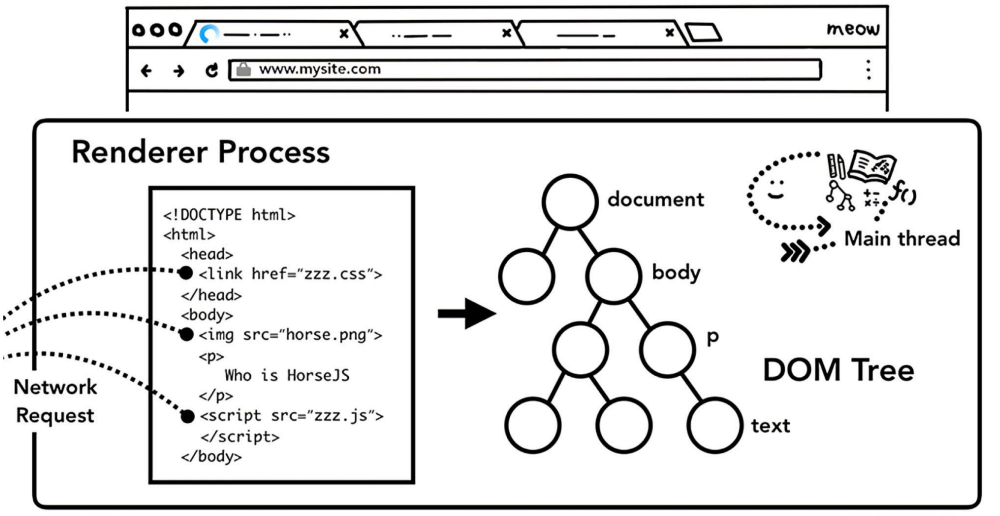
You can check out the use cases of each style in the diagram.

Source: https://lnkd.in/gFgi33RY I combined a few diagrams together. The credit all goes to AltexSoft.

—

Check out our bestselling system design books.
Paperback: Amazon Digital: ByteByteGo.

How do modern browsers work?



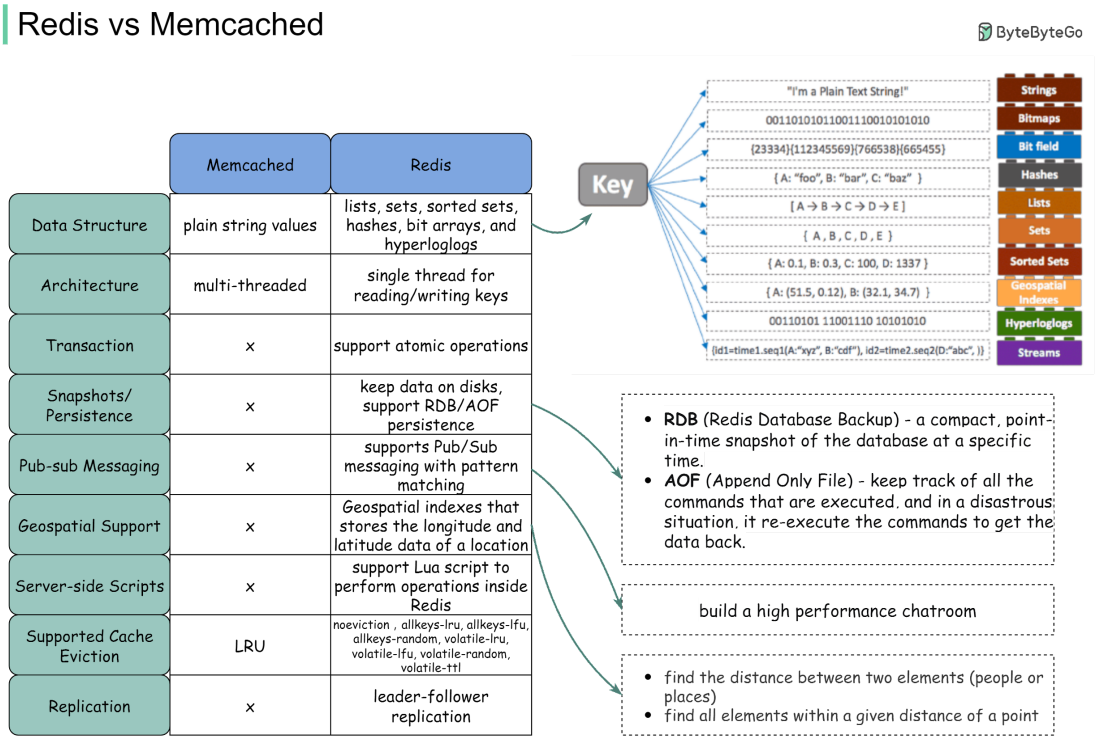
Google published a series of articles about "Inside look at modern web browser". It's a great read.

Links:
https://developer.chrome.com/blog/inside-browser-part1/
https://developer.chrome.com/blog/inside-browser-part2/
https://developer.chrome.com/blog/inside-browser-part3/
https://developer.chrome.com/blog/inside-browser-part4/

Redis vs Memcached

The diagram below illustrates the key differences.

Redis vs Memcached



The advantages on data structures make Redis a good choice for:

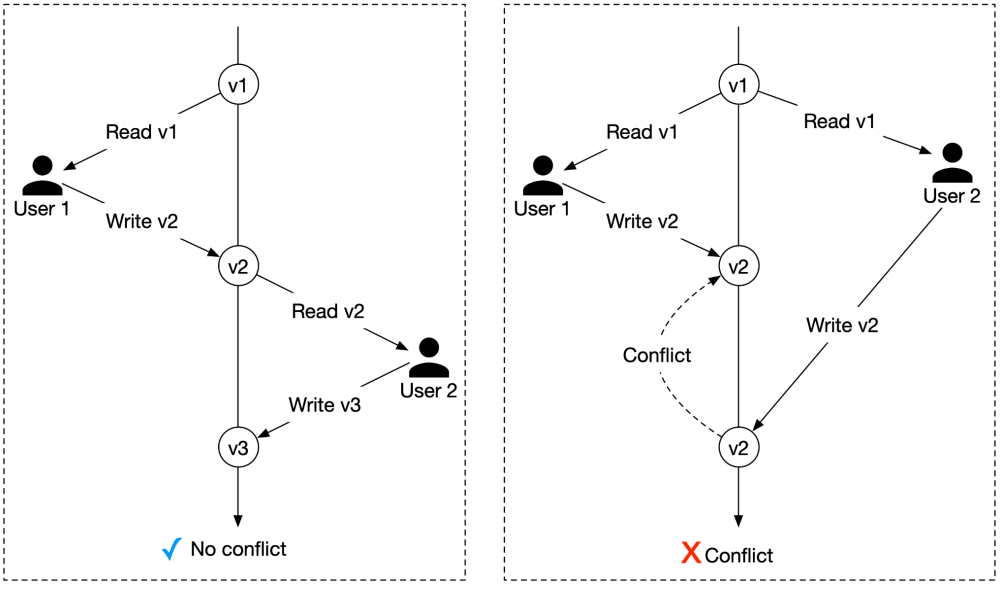
- Recording the number of clicks and comments for each post (hash)
- Sorting the commented user list and deduping the users (zset)
- Caching user behavior history and filtering malicious behaviors (zset, hash)
- Storing boolean information of extremely large data into small space. For example, login status, membership status. (bitmap)

Optimistic locking

Optimistic locking, also referred to as optimistic concurrency control, allows multiple concurrent users to attempt to update the same resource.

There are two common ways to implement optimistic locking: version number and timestamp. Version number is generally considered to be a better option because the server clock can be inaccurate over time. We explain how optimistic locking works with version number.

The diagram below shows a successful case and a failure case.



1. A new column called “version” is added to the database table.

2. Before a user modifies a database row, the application reads the version number of the row.

3. When the user updates the row, the application increases the version number by 1 and writes it back to the database.

4. A database validation check is put in place; the next version number should exceed the current version number by 1. The transaction aborts if the validation fails and the user tries again from step 2.

Optimistic locking is usually faster than pessimistic locking because we do not lock the database. However, the performance of optimistic locking drops dramatically when concurrency is high.

To understand why, consider the case when many clients try to reserve a hotel room at the same time. Because there is no limit on how many clients can read the available room count, all of them read back the same available room count and the current version number. When different clients make reservations and write back the results to the database, only one of them will succeed, and the rest of the clients receive a version check failure message. These clients have to retry. In the subsequent round of retries, there is only one successful client again, and the rest have to retry. Although the end result is correct, repeated retries cause a very unpleasant user experience.

Question: what are the possible ways of solving race conditions?

Tradeoff between latency and consistency

Understanding the **tradeoffs** is very important not only in system design interviews but also designing real-world systems. When we talk about data replication, there is a fundamental tradeoff between **latency** and **consistency**. It is illustrated by the diagram below.

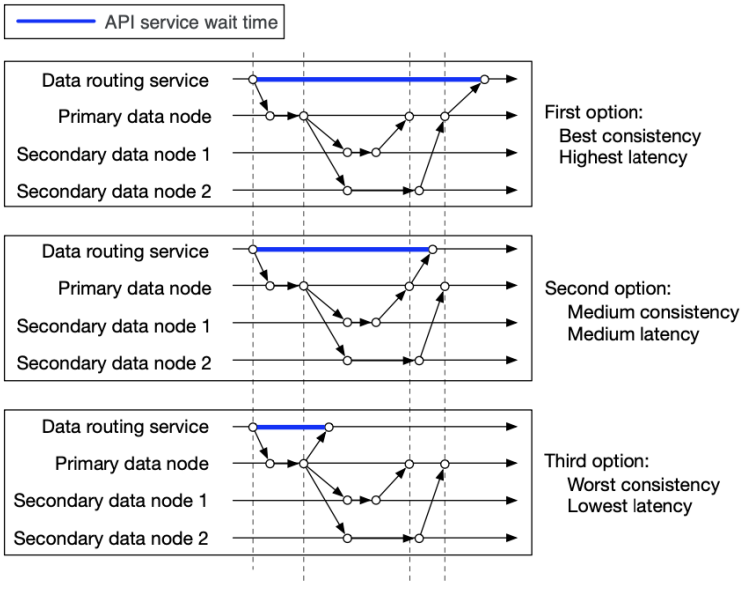


Figure 9.11: Trade-off between consistency and latency

1. Data is considered as successfully saved after all three nodes store the data. This approach has the best consistency but the highest latency.
2. Data is considered as successfully saved after the primary and one of the secondaries store the data. This approach has a medium consistency and medium latency.
3. Data is considered as successfully saved after the primary persists the data. This approach has the worst consistency but the lowest latency.

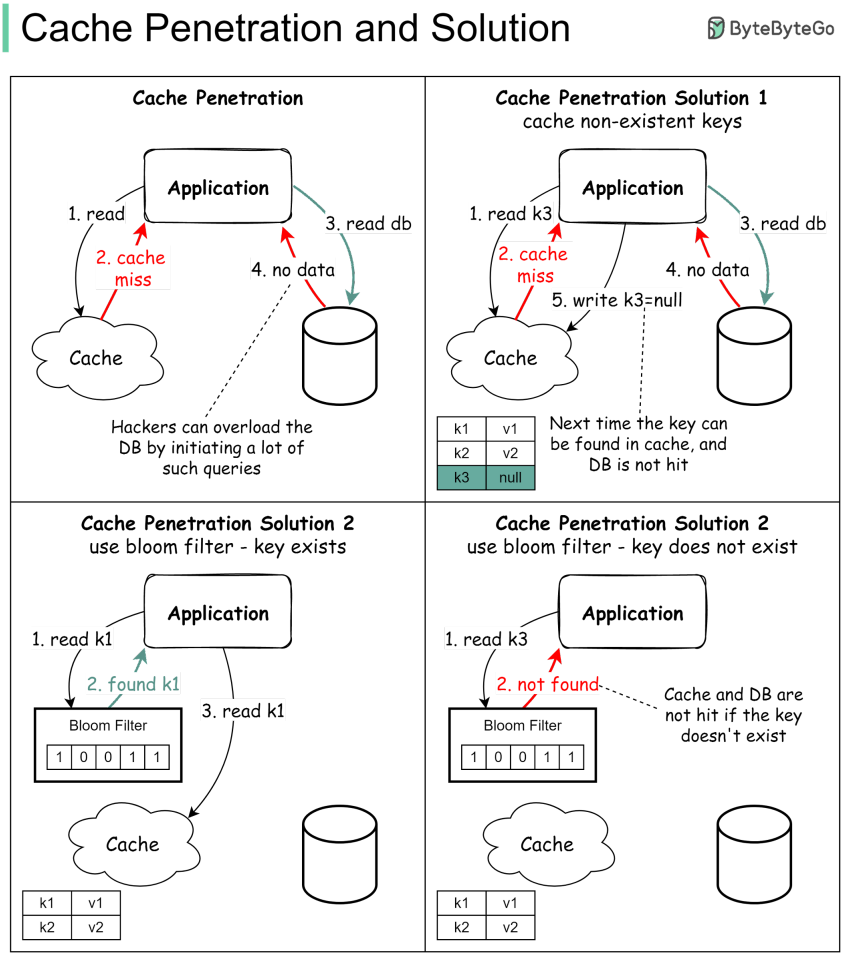
Both 2 and 3 are forms of eventual consistency.

Cache miss attack

Caching is awesome but it doesn’t come without a cost, just like many things in life.

One of the issues is **Cache Miss Attack**. Correct me if this is not the right term. It refers to the scenario where data to fetch doesn't exist in the database and the data isn’t cached either. So every request hits the database eventually, defeating the purpose of using a cache. If a malicious user initiates lots of queries with such keys, the database can easily be overloaded.

The diagram below illustrates the process.



Two approaches are commonly used to solve this problem:

🔹 Cache keys with null value. Set a short TTL (Time to Live) for keys with null value.

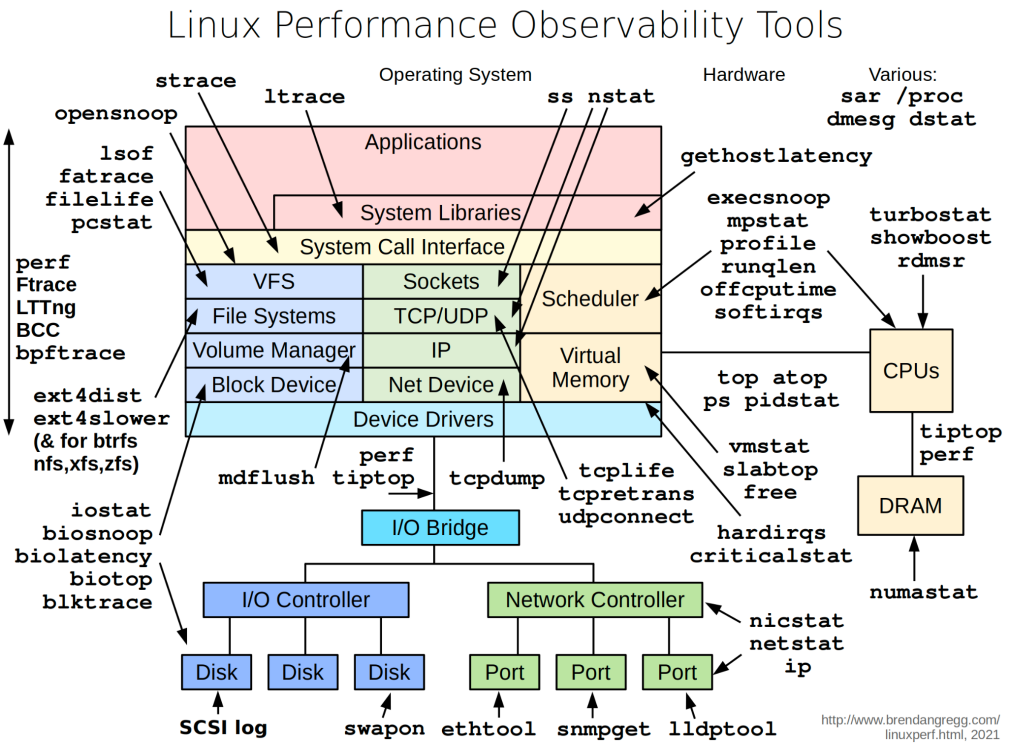
🔹 Using Bloom filter. A Bloom filter is a data structure that can rapidly tell us whether an element is present in a set or not. If the key exists, the request first goes to the cache and then queries the database if needed. If the key doesn't exist in the data set, it means the key doesn’t exist in the cache/database. In this case, the query will not hit the cache or database layer.

—

Check out our bestselling system design books.
Paperback: Amazon Digital: ByteByteGo.

How to diagnose a mysterious process that's taking too much CPU, memory, IO, etc?

The diagram below illustrates helpful tools in a Linux system.



- 'vmstat' - reports information about processes, memory, paging, block IO, traps, and CPU activity.

- 'iostat' - reports CPU and input/output statistics of the system.

- 'netstat' - displays statistical data related to IP, TCP, UDP, and ICMP protocols.

- 'lsof' - lists open files of the current system.

- 'pidstat' - monitors the utilization of system resources by all or specified processes, including CPU, memory, device IO, task switching, threads, etc.

What are the top cache strategies?

Read data from the system:

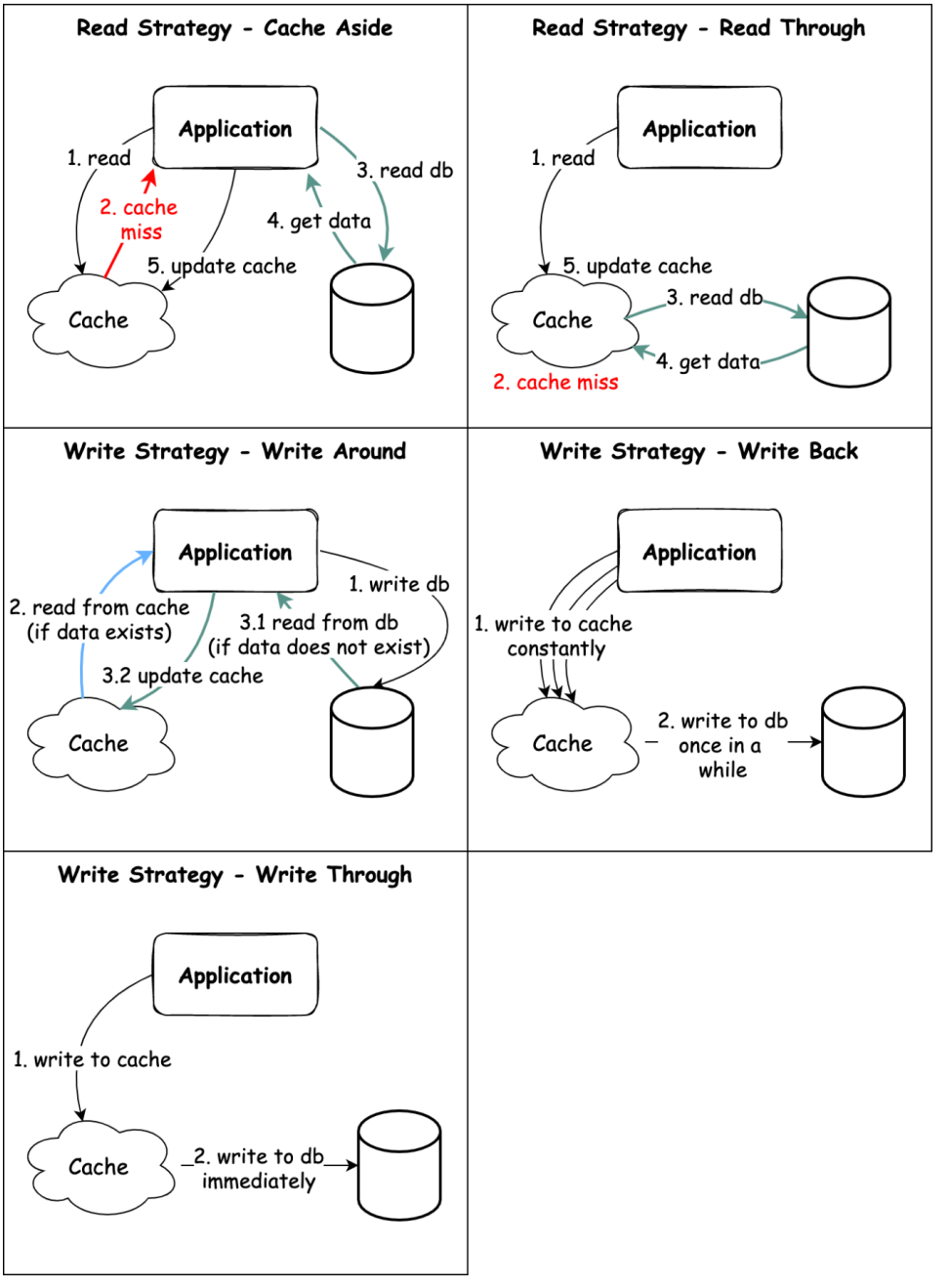
- Cache aside
- Read through

Write data to the system:

- Write around
- Write back
- Write through

The diagram below illustrates how those 5 strategies work. Some of the caching strategies can be used together.

Top caching strategies



I left out a lot of details as that will make the post very long. Feel free to leave a comment so we can learn from each other.

Question: What are the pros and cons of each caching strategy? How to choose the right one to use?

—

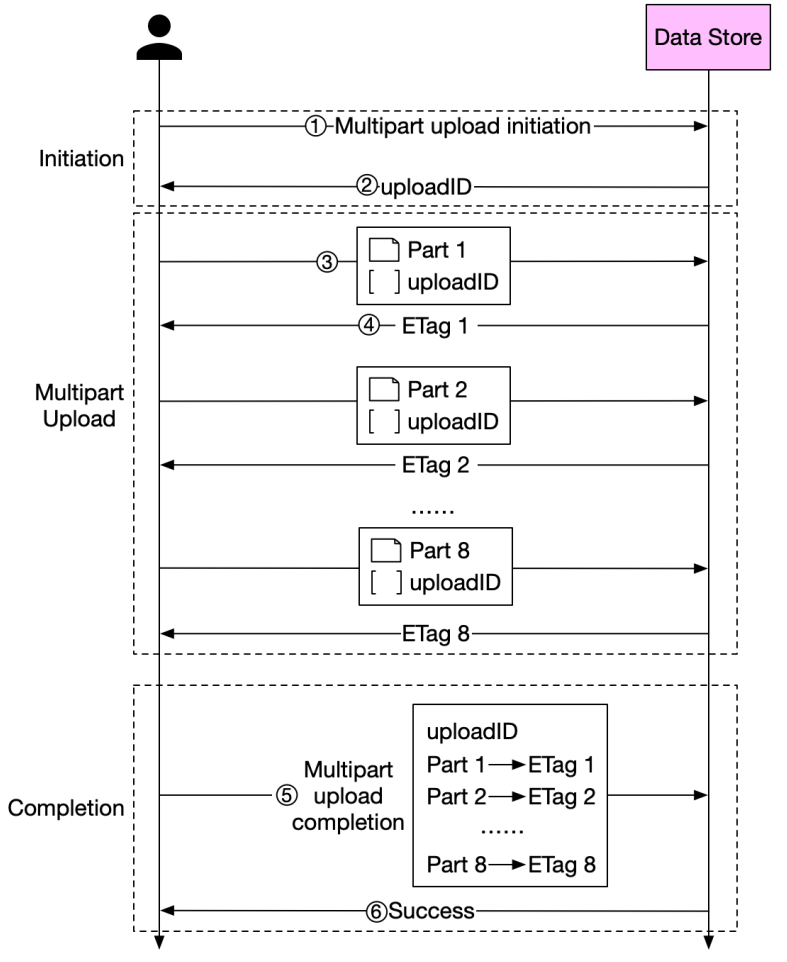
Check out our bestselling system design books.
Paperback: Amazon Digital: ByteByteGo.

Upload large files

How can we optimize performance when we **upload large files** to object storage service such as S3?

Before we answer this question, let's take a look at why we need to optimize this process. Some files might be larger than a few GBs. It is possible to upload such a large object file directly, but it could take a long time. If the network connection fails in the middle of the upload, we have to start over. A better solution is to slice a large object into smaller parts and upload them independently. After all the parts are uploaded, the object store re-assembles the object from the parts. This process is called **multipart upload**.

The diagram below illustrates how multipart upload works:



1. The client calls the object storage to initiate a multipart upload.

2. The data store returns an uploadID, which uniquely identifies the upload.

3. The client splits the large file into small objects and starts uploading. Let's assume the size of the file is 1.6GB and the client splits it into 8 parts, so each part is 200 MB in size. The client uploads the first part to the data store together with the uploadID it received in step 2.

4. When a part is uploaded, the data store returns an ETag, which is essentially the md5 checksum of that part. It is used to verify multipart uploads.

5. After all parts are uploaded, the client sends a complete multipart upload request, which includes the uploadID, part numbers, and ETags.

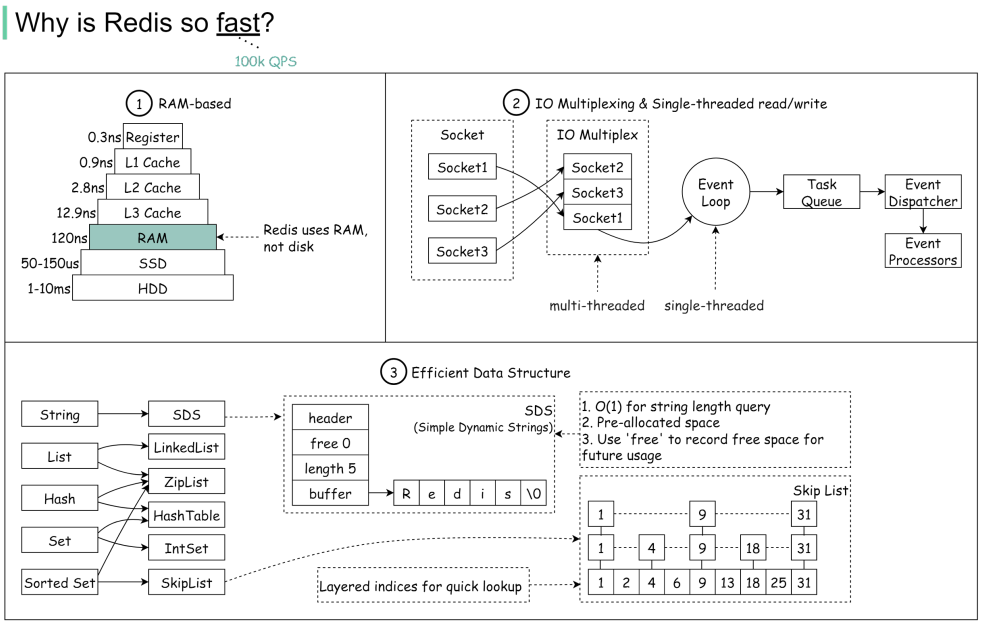
6. The data store reassembles the object from its parts based on the part number. Since the object is really large, this process may take a few minutes. After reassembly is complete, it returns a success message to the client.

—

Check out our bestselling system design books.
Paperback: Amazon Digital: ByteByteGo.

Why is Redis so Fast?

There are 3 main reasons as shown in the diagram below.



1. Redis is a RAM-based database. RAM access is at least 1000 times faster than random disk access.

2. Redis leverages IO multiplexing and single-threaded execution loop for execution efficiency.

3. Redis leverages several efficient lower-level data structures.

Question: Another popular in-memory store is Memcached. Do you know the differences between Redis and Memcached?

You might have noticed the style of this diagram is different from my previous posts. Please let me know which one you prefer.

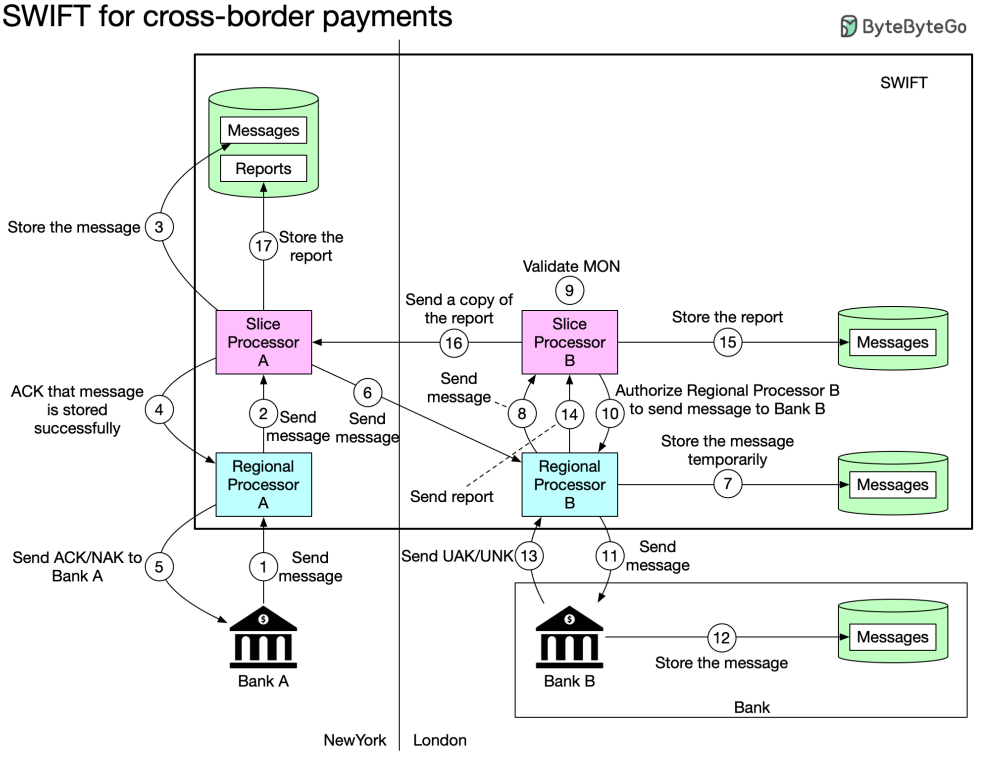
—

Check out our bestselling system design books.
Paperback: Amazon Digital: ByteByteGo.

SWIFT payment network

You probably heard about **SWIFT**. What is SWIFT? What role does it play in cross-border payments? You can find answers to those questions in this post.

SWIFT for cross-border payments



The Society for Worldwide Interbank Financial Telecommunication (SWIFT) is the main secure **messaging system** that links the world's banks.

The Belgium-based system is run by its member banks and handles millions of payment messages per day. The diagram below illustrates how payment messages are transmitted from Bank A (in New York) to Bank B (in London).

Step 1: Bank A sends a message with transfer details to Regional Processor A in New York. The destination is Bank B.

Step 2: Regional processor validates the format and sends it to Slice Processor A. The Regional Processor is responsible for input message validation and output message queuing. The Slice Processor is responsible for storing and routing messages safely.

Step 3: Slice Processor A stores the message.

Step 4: Slice Processor A informs Regional Processor A the message is stored.

Step 5: Regional Processor A sends ACK/NAK to Bank A. ACK means a message will be sent to Bank B. NAK means the message will NOT be sent to Bank B.

Step 6: Slice Processor A sends the message to Regional Processor B in London.

Step 7: Regional Processor B stores the message temporarily.

Step 8: Regional Processor B assigns a unique ID MON (Message Output Number) to the message and sends it to Slice Processor B

Step 9: Slice Processor B validates MON.

Step 10: Slice Processor B authorizes Regional Processor B to send the message to Bank B.

Step 11: Regional Processor B sends the message to Bank B.

Step 12: Bank B receives the message and stores it.

Step 13: Bank B sends UAK/UNK to Regional Processor B. UAK (user positive acknowledgment) means Bank B received the message without error; UNK (user negative acknowledgment) means Bank B received checksum failure.

Step 14: Regional Processor B creates a report based on Bank B’s response, and sends it to Slice Processor B.

Step 15: Slice Processor B stores the report.

Step 16 - 17: Slice Processor B sends a copy of the report to Slice Processor A. Slice Processor A stores the report.

—

Check out our bestselling system design books.
Paperback: Amazon Digital: ByteByteGo.

At-most once, at-least once, and exactly once

In modern architecture, systems are broken up into small and independent building blocks with well-defined interfaces between them. Message queues provide communication and coordination for those building blocks. Today, let's discuss different delivery semantics: at-most once, at-least once, and exactly once.

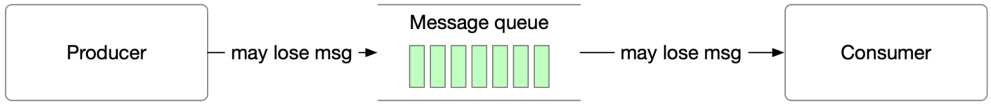


Figure 1 At-most once

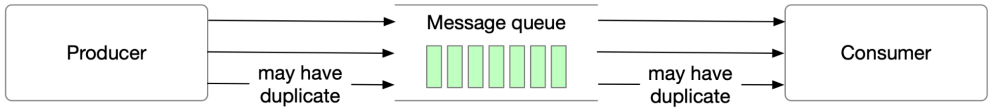


Figure 2 At-least once

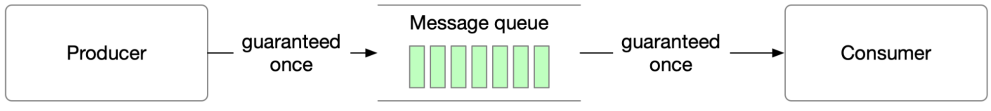


Figure 3 Exactly-once

At-most once

As the name suggests, at-most once means a message will be delivered not more than once. Messages may be lost but are not redelivered. This is how at-most once delivery works at the high level.

Use cases: It is suitable for use cases like monitoring metrics, where a small amount of data loss is acceptable.

At-least once

With this data delivery semantic, it's acceptable to deliver a message more than once, but no message should be lost.

Use cases: With at-least once, messages won't be lost but the same message might be delivered multiple times. While not ideal from a user perspective, at-least once delivery semantics are usually good enough for use cases where data duplication is not a big issue or deduplication

is possible on the consumer side. For example, with a unique key in each message, a message can be rejected when writing duplicate data to the database.

Exactly once

Exactly once is the most difficult delivery semantic to implement. It is friendly to users, but it has a high cost for the system's performance and complexity.

Use cases: Financial-related use cases (payment, trading, accounting, etc.). Exactly once is especially important when duplication is not acceptable and the downstream service or third party doesn't support idempotency.

Question: what is the difference between message queues vs event streaming platforms such as Kafka, Apache Pulsar, etc?

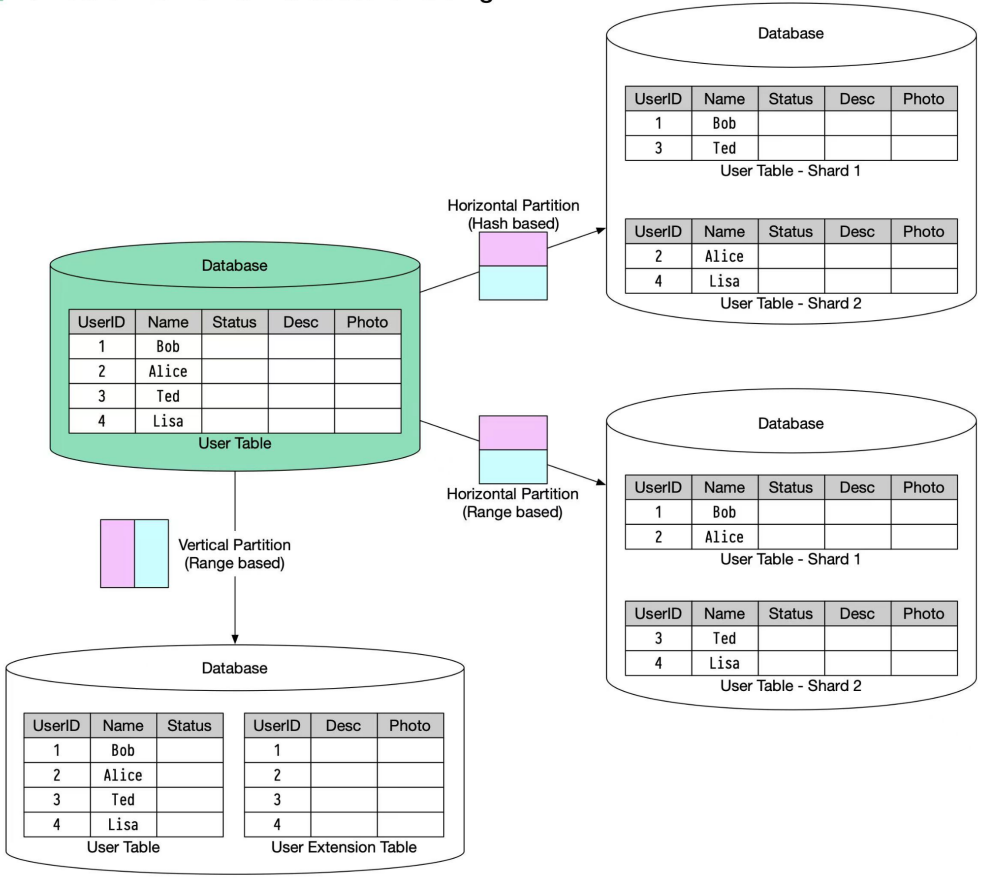
Vertical partitioning and Horizontal partitioning

In many large-scale applications, data is divided into partitions that can be accessed separately. There are two typical strategies for partitioning data.

- Vertical partitioning: it means some columns are moved to new tables. Each table contains the same number of rows but fewer columns (see diagram below).

- Horizontal partitioning (often called sharding): it divides a table into multiple smaller tables. Each table is a separate data store, and it contains the same number of columns, but fewer rows (see diagram below).

Vertical & Horizontal Database Sharding



Horizontal partitioning is widely used so let’s take a closer look.

Routing algorithm
Routing algorithm decides which partition (shard) stores the data.

- Range-based sharding. This algorithm uses ordered columns, such as integers, longs, timestamps, to separate the rows. For example, the diagram below uses the User ID column for range partition: User IDs 1 and 2 are in shard 1, User IDs 3 and 4 are in shard 2.

- Hash-based sharding. This algorithm applies a hash function to one column or several columns to decide which row goes to which table. For example, the diagram below uses **User ID mod** 2 as a hash function. User IDs 1 and 3 are in shard 1, User IDs 2 and 4 are in shard 2.

Benefits
- Facilitate horizontal scaling. Sharding facilitates the possibility of adding more machines to spread out the load.

- Shorten response time. By sharding one table into multiple tables, queries go over fewer rows, and results are returned much more quickly.

Drawbacks
- The order by the operation is more complicated. Usually, we need to fetch data from different shards and sort the data in the application's code.

- Uneven distribution. Some shards may contain more data than others (this is also called the hotspot).

This topic is very big and I’m sure I missed a lot of important details. What else do you think is important for data partitioning?

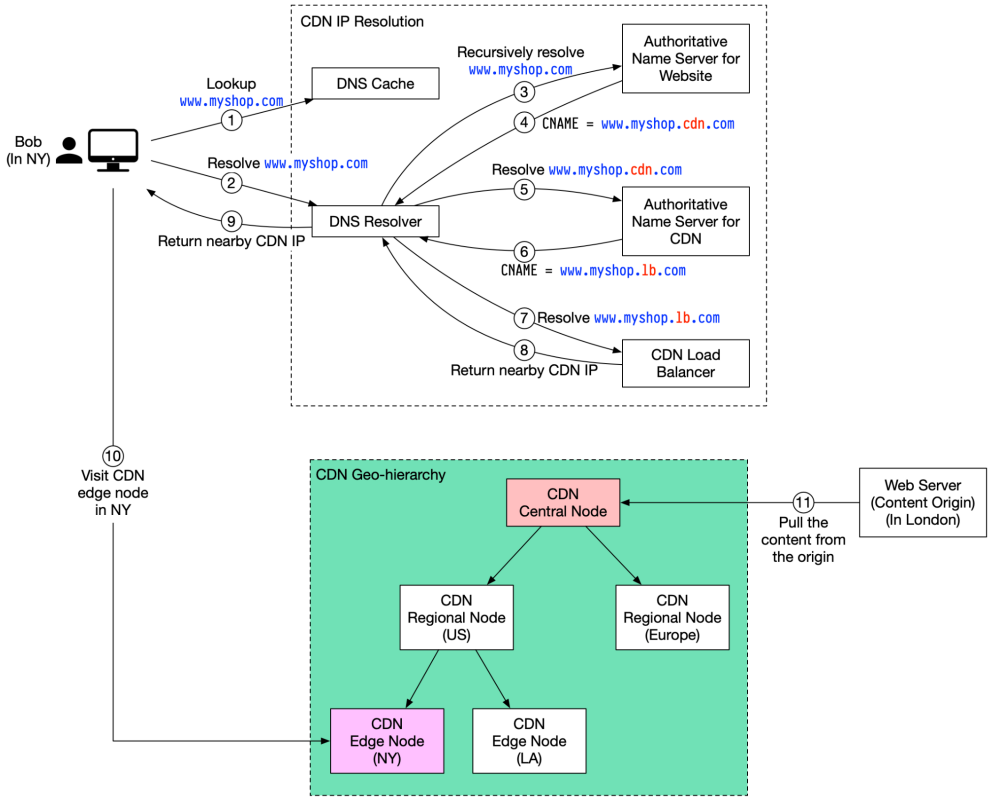
CDN

A content delivery network (CDN) refers to a geographically distributed servers (also called edge servers) which provide fast delivery of static and dynamic content. Let's take a look at how it works.

Suppose Bob who lives in New York wants to visit an eCommerce website that is deployed in London. If the request goes to servers located in London, the response will be quite slow. So we deploy CDN servers close to where Bob lives, and the content will be loaded from the nearby CDN server.

The diagram below illustrates the process:

How does CDN work



1. Bob types in www.myshop.com in the browser. The browser looks up the domain name in the local DNS cache.

2. If the domain name does not exist in the local DNS cache, the browser goes to the DNS resolver to resolve the name. The DNS resolver usually sits in the Internet Service Provider (ISP).

3. The DNS resolver recursively resolves the domain name (see my previous post for details). Finally, it asks the authoritative name server to resolve the domain name.

4. If we don’t use CDN, the authoritative name server returns the IP address for www.myshop.com. But with CDN, the authoritative name server has an alias pointing to www.myshop.cdn.com (the domain name of the CDN server).

5. The DNS resolver asks the authoritative name server to resolve www.myshop.cdn.com.

6. The authoritative name server returns the domain name for the load balancer of CDN www.myshop.lb.com.

7. The DNS resolver asks the CDN load balancer to resolve www.myshop.lb.com. The load balancer chooses an optimal CDN edge server based on the user’s IP address, user’s ISP, the content requested, and the server load.

8. The CDN load balancer returns the CDN edge server’s IP address for www.myshop.lb.com.

9. Now we finally get the actual IP address to visit. The DNS resolver returns the IP address to the browser.

10. The browser visits the CDN edge server to load the content. There are two types of contents cached on the CDN servers: static contents and dynamic contents. The former contains static pages, pictures, and videos; the latter one includes results of edge computing.

11. If the edge CDN server cache doesn't contain the content, it goes upward to the regional CDN server. If the content is still not found, it will go upward to the central CDN server, or even go to the origin - the

London web server. This is called the CDN distribution network, where the servers are deployed geographically.

Over to you: How do you prevent videos cached on CDN from being pirated?

Erasure coding

A really cool technique that's commonly used in object storage such as S3 to improve durability is called **Erasure Coding**. Let's take a look at how it works.

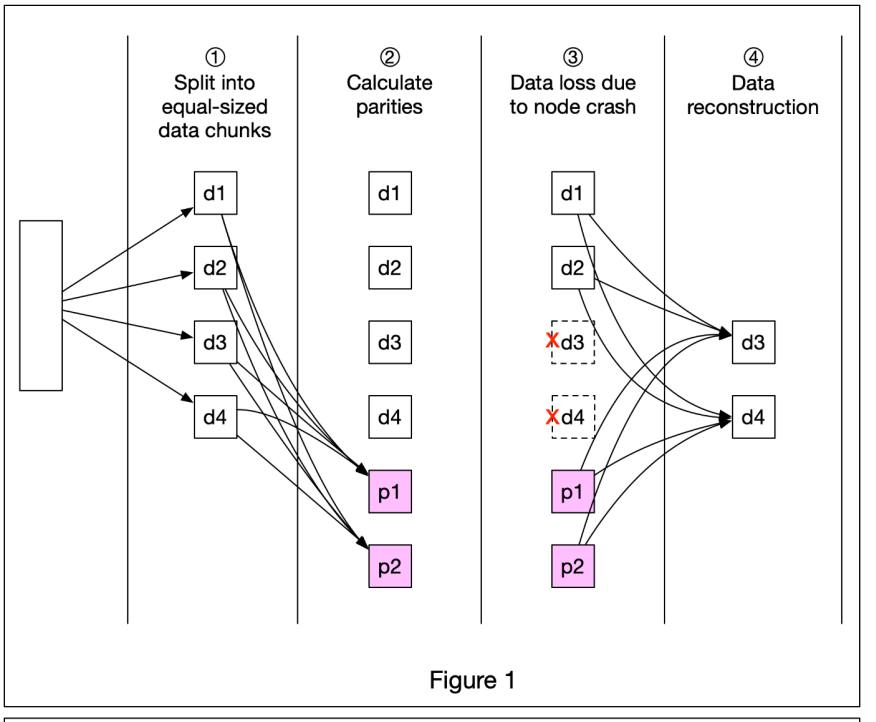


Figure 1

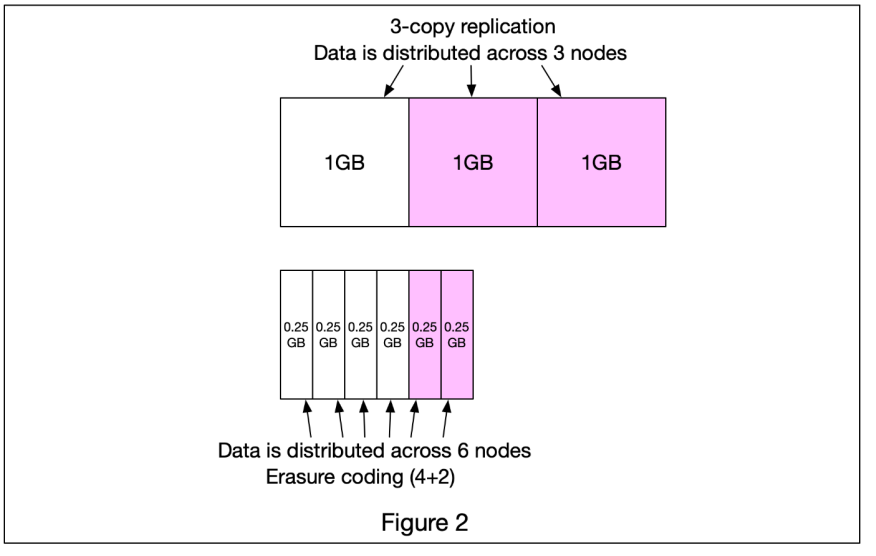


Figure 2

Erasure coding deals with data durability differently from replication. It chunks data into smaller pieces (placed on different servers) and creates parities for redundancy. In the event of failures, we can use chunk data and parities to reconstruct the data. Let's take a look at a concrete example (4 + 2 erasure coding) as shown in Figure 1.

1 Data is broken up into four even-sized data chunks d1, d2, d3, and d4.

2 The mathematical formula is used to calculate the parities p1 and p2. To give a much simplified example, p1 = d1 + 2*d2 - d3 + 4*d4 and p2 = -d1 + 5*d2 + d3 - 3*d4.

3 Data d3 and d4 are lost due to node crashes.

4 The mathematical formula is used to reconstruct lost data d3 and d4, using the known values of d1, d2, p1, and p2.

How much extra space does erasure coding need? For every two chunks of data, we need one parity block, so the storage overhead is 50% (Figure 2). While in 3-copy replication, the storage overhead is 200% (Figure 2).

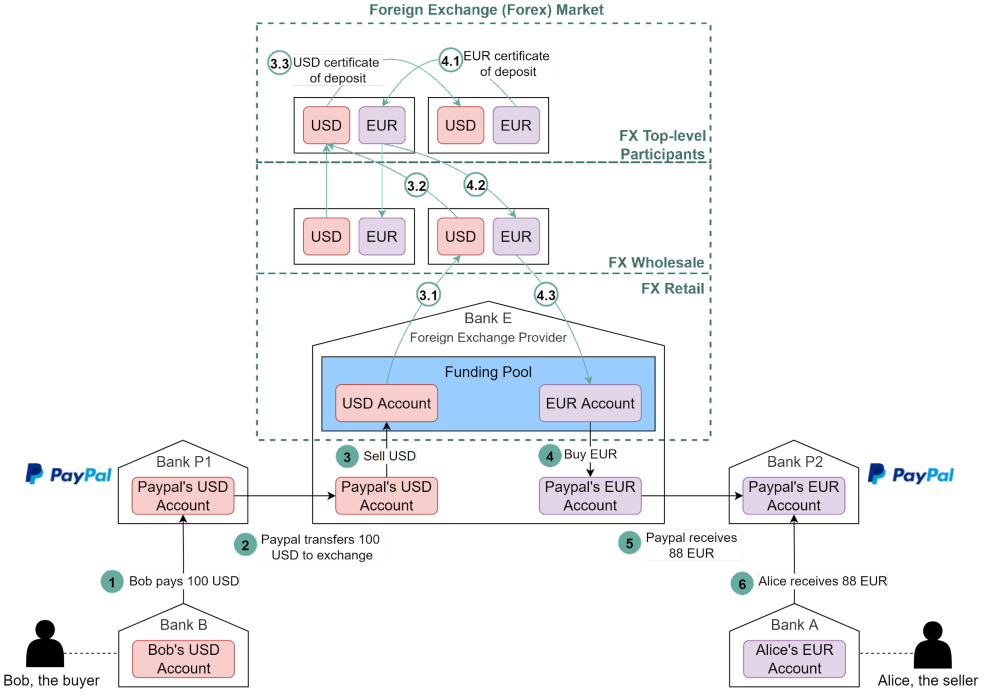
Does erasure coding increase data durability? Let's assume a node has a 0.81% annual failure rate. According to the calculation done by Backblaze, erasure coding can achieve 11 nines durability vs 3-copy replication can achieve 6 nines durability.

What other techniques do you think are important to improve the scalability and durability of an object store such as S3?

Foreign exchange in payment

Have you wondered what happens under the hood when you pay with USD online and the seller from Europe receives EUR (euro)? This process is called foreign exchange.

Foreign Exchange in Payments



Suppose Bob (the buyer) needs to pay 100 USD to Alice (the seller), and Alice can only receive EUR. The diagram below illustrates the process.

1. Bob sends 100 USD via a third-party payment provider. In our example, it is Paypal. The money is transferred from Bob's bank account (Bank B) to Paypal's account in Bank P1.

2. Paypal needs to convert USD to EUR. It leverages the foreign exchange provider (Bank E). Paypal sends 100 USD to its USD account in Bank E.

3. 100 USD is sold to Bank E's funding pool.

4. Bank E's funding pool provides 88 EUR in exchange for 100 USD. The money is put into Paypal's EUR account in Bank E.

5. Paypal's EUR account in Bank P2 receives 88 EUR.

6. 88 EUR is paid to Alice's EUR account in Bank A.

Now let's take a close look at the foreign exchange (forex) market. It has 3 layers:

- Retail market. Funding pools are parts of the retail market. To improve efficiency, Paypal usually buys a certain amount of foreign currencies in advance.
- Wholesale market. The wholesale business is composed of investment banks, commercial banks, and foreign exchange providers. It usually handles accumulated orders from the retail market.
- Top-level participants. They are multinational commercial banks that hold a large number of certificates of deposit from different countries. They exchange these certificates for foreign exchange trading.

When Bank E's funding pool needs more EUR, it goes upward to the wholesale market to sell USD and buy EUR. When the wholesale market accumulates enough orders, it goes upward to top-level participants. Steps 3.1-3.3 and 4.1-4.3 explain how it works.

If you have any questions, please leave a comment.

What foreign currency did you find difficult to exchange? And what company have you used for foreign currency exchange?

Interview Question: Design S3

What happens when you upload a file to Amazon S3? Let's design an S3 like object storage system.

Upload a File to S3

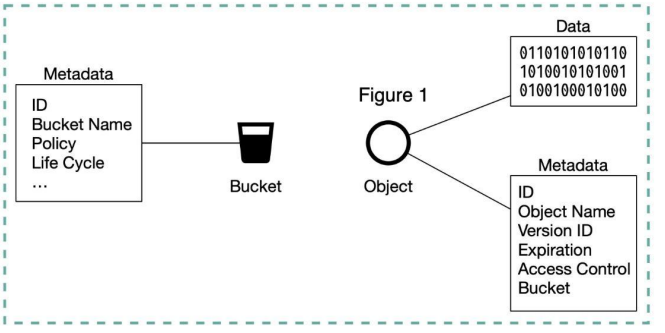


Figure 1

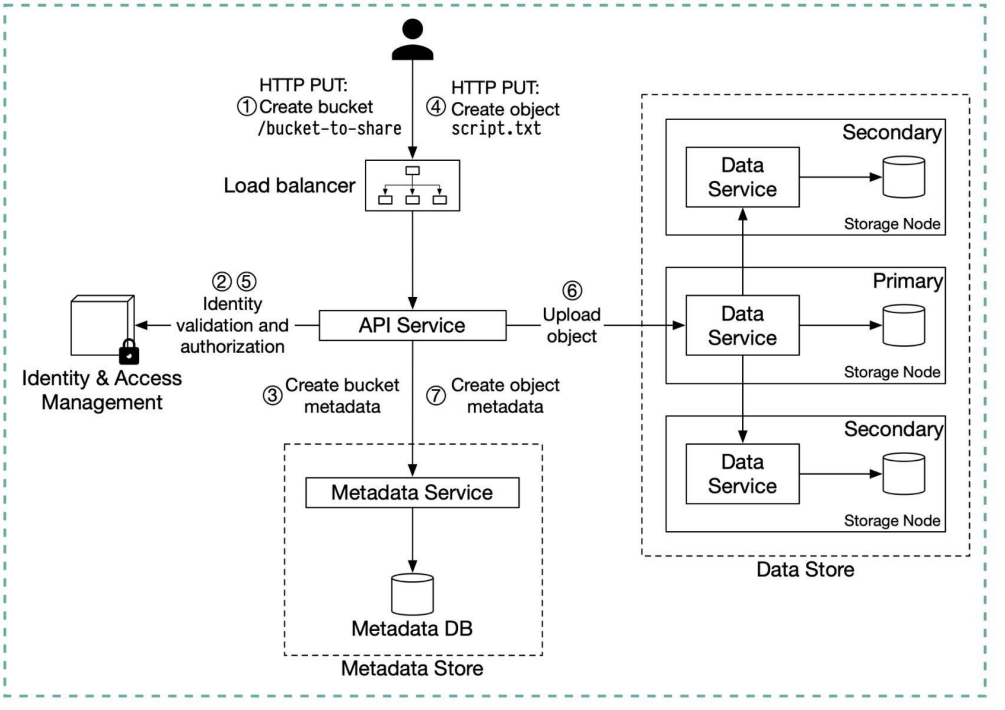


Figure 2

Before we dive into the design, let's define some terms.

Bucket. A logical container for objects. The bucket name is globally unique. To upload data to S3, we must first create a bucket.

Object. An object is an individual piece of data we store in a bucket. It contains object data (also called payload) and metadata. Object data can be any sequence of bytes we want to store. The metadata is a set of name-value pairs that describe the object.

An S3 object consists of (Figure 1):

- Metadata. It is mutable and contains attributes such as ID, bucket name, object name, etc.
- Object data. It is immutable and contains the actual data.

In S3, an object resides in a bucket. The path looks like this: /bucket-to-share/script.txt. The bucket only has metadata. The object has metadata and the actual data.

The diagram below (Figure 2) illustrates how file uploading works. In this example, we first create a bucket named “bucket-to-share” and then upload a file named “script.txt” to the bucket.

1. The client sends an HTTP PUT request to create a bucket named “bucket-to-share.” The request is forwarded to the API service.

2. The API service calls the Identity and Access Management (IAM) to ensure the user is authorized and has WRITE permission.

3. The API service calls the metadata store to create an entry with the bucket info in the metadata database. Once the entry is created, a success message is returned to the client.

4. After the bucket is created, the client sends an HTTP PUT request to create an object named “script.txt”.

5. The API service verifies the user’s identity and ensures the user has WRITE permission on the bucket.

6. Once validation succeeds, the API service sends the object data in the HTTP PUT payload to the data store. The data store persists the payload as an object and returns the UUID of the object.

7. The API service calls the metadata store to create a new entry in the metadata database. It contains important metadata such as the object_id (UUID), bucket_id (which bucket the object belongs to), object_name, etc.

Block storage, file storage and object storage

Yesterday, I posted the definitions of block storage, file storage, and object storage. Let's continue the discussion and compare those 3 options.

	Block storage	File storage	Object storage
Mutable Content	Y	Y	N (object versioning is supported, in-place update is not)
Cost	High	Medium to high	Low
Performance	Medium to high, very high	Medium to high	Low to medium
Consistency	Strong consistency	Strong consistency	Strong consistency
Data access	SAS/iSCSI/FC	Standard file access, CIFS/SMB, and NFS	RESTful API
Scalability	Medium scalability	High scalability	Vast scalability
Good for	Virtual machines (VM), high-performance applications like database	General-purpose file system access	Binary data, unstructured data

Table 1 Storage options

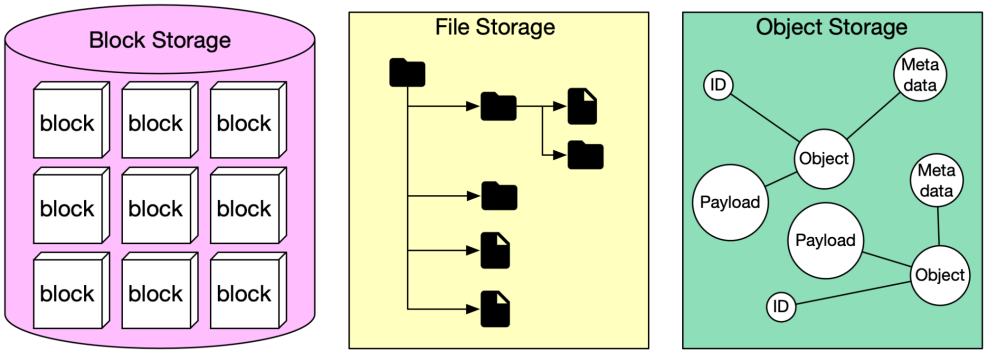
Block storage, file storage and object storage

In this post, let's review the storage systems in general.

Storage systems fall into three broad categories:

- Block storage
- File storage
- Object storage

The diagram below illustrates the comparison of different storage systems.



Block storage

Block storage came first, in the 1960s. Common storage devices like hard disk drives (HDD) and solid-state drives (SSD) that are physically attached to servers are all considered as block storage.

Block storage presents the raw blocks to the server as a volume. This is the most flexible and versatile form of storage. The server can format the raw blocks and use them as a file system, or it can hand control of those blocks to an application. Some applications like a database or a virtual machine engine manage these blocks directly in order to squeeze every drop of performance out of them.

Block storage is not limited to physically attached storage. Block storage could be connected to a server over a high-speed network or over industry-standard connectivity protocols like Fibre Channel (FC)

and iSCSI. Conceptually, the network-attached block storage still presents raw blocks. To the servers, it works the same as physically attached block storage. Whether to a network or physically attached, block storage is fully owned by a single server. It is not a shared resource.

File storage

File storage is built on top of block storage. It provides a higher-level abstraction to make it easier to handle files and directories. Data is stored as files under a hierarchical directory structure. File storage is the most common general-purpose storage solution. File storage could be made accessible by a large number of servers using common file-level network protocols like SMB/CIFS and NFS. The servers accessing file storage do not need to deal with the complexity of managing the blocks, formatting volume, etc. The simplicity of file storage makes it a great solution for sharing a large number of files and folders within an organization.

Object storage

Object storage is new. It makes a very deliberate tradeoff to sacrifice performance for high durability, vast scale, and low cost. It targets relatively “cold” data and is mainly used for archival and backup. Object storage stores all data as objects in a flat structure. There is no hierarchical directory structure. Data access is normally provided via a RESTful API. It is relatively slow compared to other storage types. Most public cloud service providers have an object storage offering, such as AWS S3, Google block storage, and Azure blob storage.

Domain Name System (DNS) lookup

DNS acts as an address book. It translates human-readable domain names (google.com) to machine-readable IP addresses (142.251.46.238).

To achieve better scalability, the DNS servers are organized in a hierarchical tree structure.

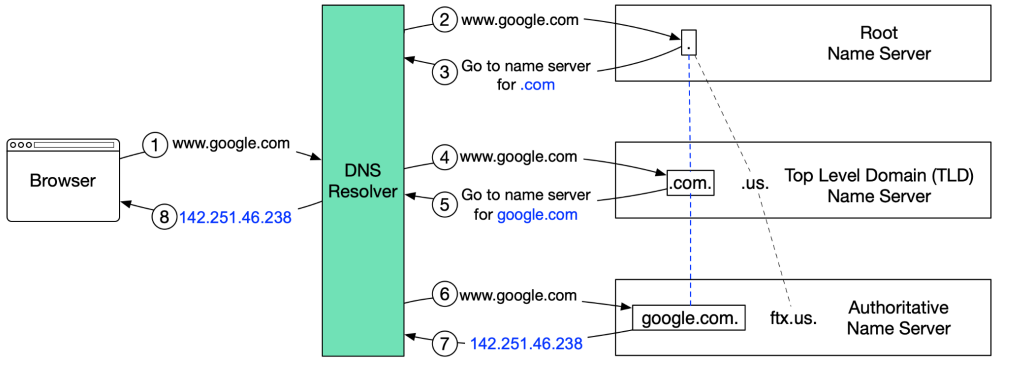
There are 3 basic levels of DNS servers:

1. Root name server (.). It stores the IP addresses of Top Level Domain (TLD) name servers. There are 13 logical root name servers globally.
2. TLD name server. It stores the IP addresses of authoritative name servers. There are several types of TLD names. For example, generic TLD (.com, .org), country code TLD (.us), test TLD (.test).

3. Authoritative name server. It provides actual answers to the DNS query. You can register authoritative name servers with domain name registrar such as GoDaddy, Namecheap, etc.

The diagram below illustrates how DNS lookup works under the hood:

How does DNS resolve IP



1. google.com is typed into the browser, and the browser sends the domain name to the DNS resolver.

2. The resolver queries a DNS root name server.

3. The root server responds to the resolver with the address of a TLD DNS server. In this case, it is .com.

4. The resolver then makes a request to the .com TLD.

5. The TLD server responds with the IP address of the domain's name server, google.com (authoritative name server).

6. The DNS resolver sends a query to the domain's nameserver.

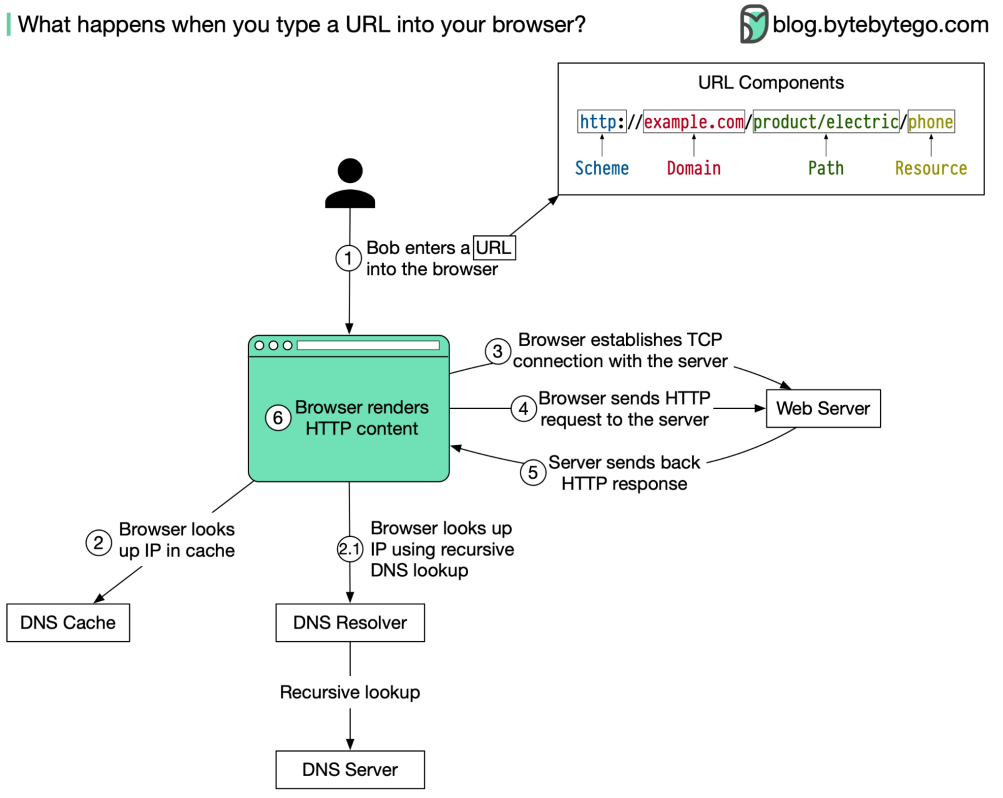
7. The IP address for google.com is then returned to the resolver from the nameserver.

8. The DNS resolver responds to the web browser with the IP address (142.251.46.238) of the domain requested initially.

DNS lookups on average take between 20-120 milliseconds to complete (according to YSlow).

What happens when you type a URL into your browser?

The diagram below illustrates the steps.



1. Bob enters a URL into the browser and hits Enter. In this example, the URL is composed of 4 parts:

- scheme - ***https***://. This tells the browser to send a connection to the server using HTTPS.
- domain - ***example.com***. This is the domain name of the site.
- path - ***product/electric***. It is the path on the server to the requested resource: phone.
- resource - ***phone***. It is the name of the resource Bob wants to visit.

2. The browser looks up the IP address for the domain with a domain name system (DNS) lookup. To make the lookup process fast, data is cached at different layers: browser cache, OS cache, local network cache and ISP cache.

2.1 If the IP address cannot be found at any of the caches, the browser goes to DNS servers to do a recursive DNS lookup until the IP address is found (this will be covered in another post).

3. Now that we have the IP address of the server, the browser establishes a TCP connection with the server.

4. The browser sends a HTTP request to the server. The request looks like this:

```
GET /phone HTTP/1.1
Host: example.com
```

5. The server processes the request and sends back the response. For a successful response (the status code is 200). The HTML response might look like this:

```
HTTP/1.1 200 OK
Date: Sun, 30 Jan 2022 00:01:01 GMT
Server: Apache
Content-Type: text/html; charset=utf-8

<!DOCTYPE html>
<html lang="en">
Hello world
</html>
```

6. The browser renders the HTML content.

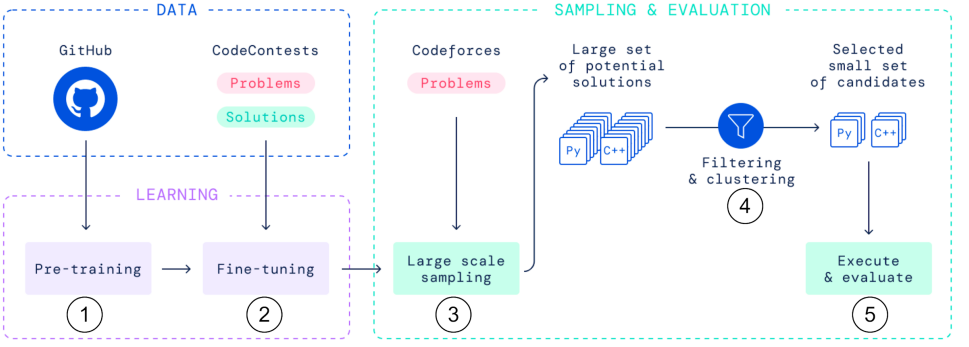
AI Coding engine

DeepMind says its new AI coding engine (AlphaCode) is as good as an average programmer.

The AI bot participated in the 10 Codeforces coding competitions and was ranked 54.3%. It means its score exceeded half of the human contestants. If we look at its score for the last 6 months, AlphaCode ranks at 28%.

The diagram below explains how the AI bot works:

How does AlphaCode Work?



Source: DeepMind Blog

1. Pre-train the transformer models on GitHub code.

2. Fine-tune the models on the relatively small competitive programming dataset.

3. At evaluation time, create a massive amount of solutions for each problem.

4. Filter, cluster and rerank the solutions to a small set of candidate programs (at most 10), and then submit for further assessments.

5. Run the candidate programs against the test cases, evaluate the performance, and choose the best one.

Do you think AI bot will be better at Leetcode or competitive programming than software engineers five years from now?

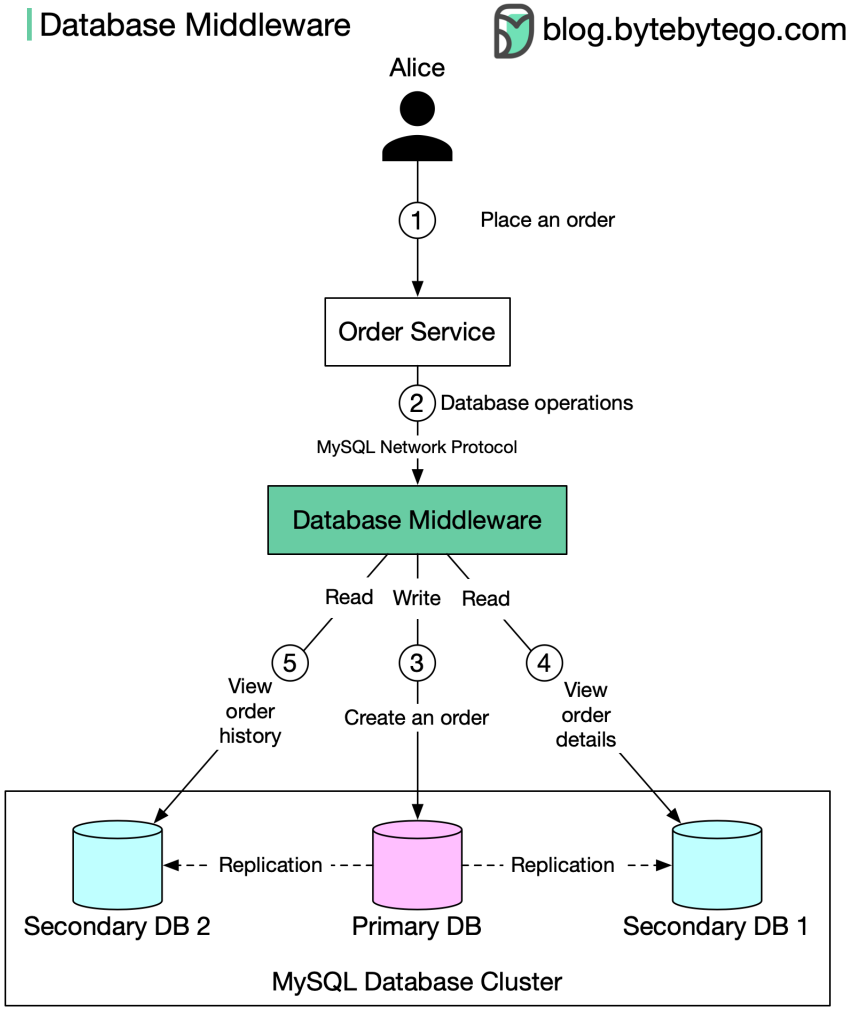
Read replica pattern

There are two common ways to implement the read replica pattern:

1. Embed the routing logic in the application code (explained in the last post).
2. Use database middleware.

We focus on option 2 here. The middleware provides transparent routing between the application and database servers. We can customize the routing logic based on difficult rules such as user, schema, statement, etc.

The diagram below illustrates the setup:



1. When Alice places an order on amazon, the request is sent to Order Service.

2. Order Service does not directly interact with the database. Instead, it sends database queries to the database middleware.

3. The database middleware routes writes to the primary database. Data is replicated to two replicas.

4. Alice views the order details (read). The request is sent through the middleware.

5. Alice views the recent order history (read). The request is sent through the middleware.

The database middleware acts as a proxy between the application and databases. It uses standard MySQL network protocol for communication.

Pros:
- Simplified application code. The application doesn't need to be aware of the database topology and manage access to the database directly.

- Better compatibility. The middleware uses the MySQL network protocol. Any MySQL compatible client can connect to the middleware easily. This makes database migration easier.

Cons:
- Increased system complexity. A database middleware is a complex system. Since all database queries go through the middleware, it usually requires a high availability setup to avoid a single point of failure.

- Additional middleware layer means additional network latency. Therefore, this layer requires excellent performance.

Read replica pattern

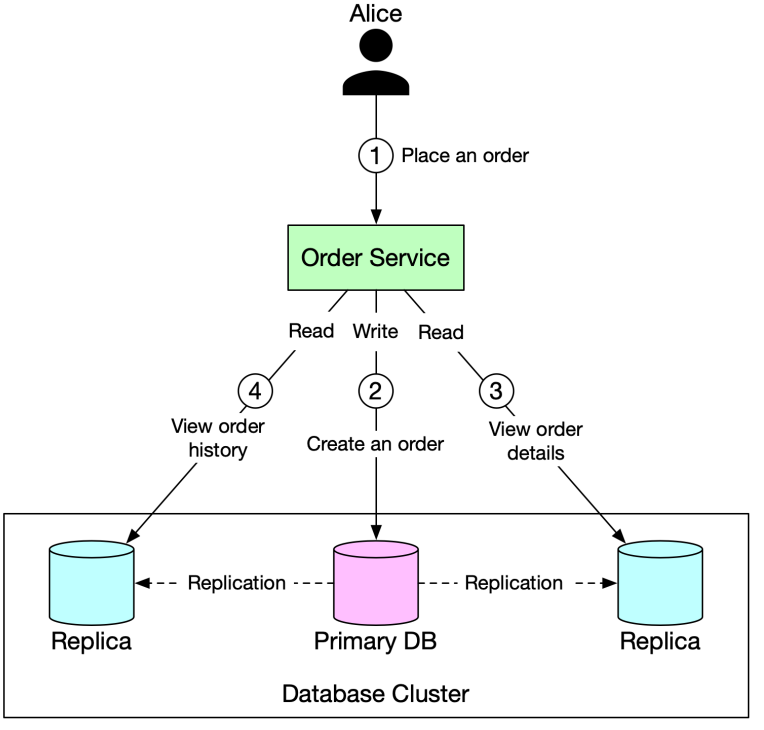
In this post, we talk about a simple yet commonly used database design pattern (setup): **Read replica pattern**.

In this setup, all data-modifying commands like insert, delete, or update are sent to the primary DB, and reads are sent to read replicas.

The diagram below illustrates the setup:

1. When Alice places an order on amazon.com, the request is sent to Order Service.
2. Order Service creates a record about the order in the primary DB (write). Data is replicated to two replicas.
3. Alice views the order details. Data is served from a replica (read).
4. Alice views the recent order history. Data is served from a replica (read).

Read Replica Pattern



There is one major problem in this setup: **replication lag**.

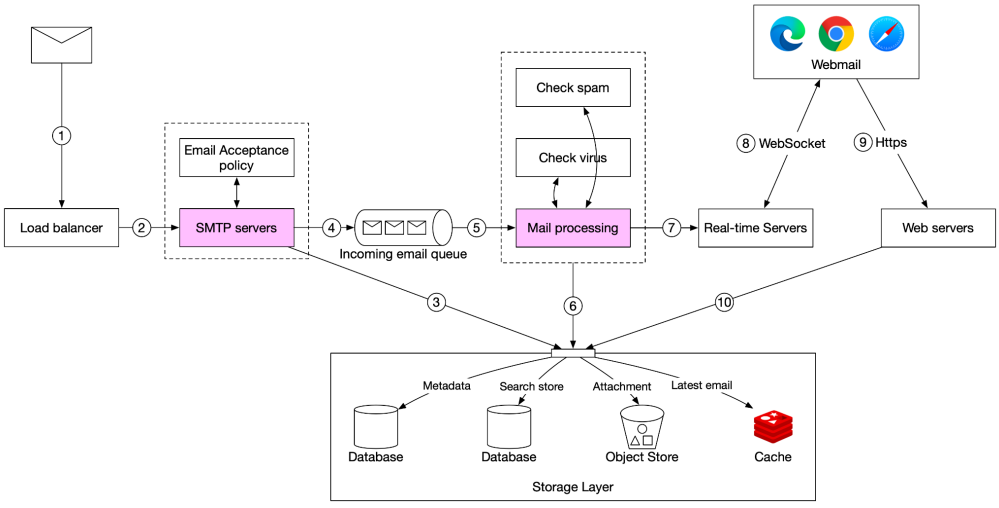
Under certain circumstances (network delay, server overload, etc.), data in replicas might be seconds or even minutes behind. In this case, if Alice immediately checks the order status (query is served by the replica) after the order is placed, she might not see the order at all. This leaves Alice confused. In this case, we need “read-after-write” consistency.

Possible solutions to mitigate this problem:

1. Latency sensitive reads are sent to the primary database.
2. Reads that immediately follow writes are routed to the primary database.
3. A relational DB generally provides a way to check if a replica is caught up with the primary. If data is up to date, query the replica. Otherwise, fail the read request or read from the primary.

Email receiving flow

The following diagram demonstrates the email receiving flow.



1. Incoming emails arrive at the SMTP load balancer.

2. The load balancer distributes traffic among SMTP servers. Email acceptance policy can be configured and applied at the SMTP-connection level. For example, invalid emails are bounced to avoid unnecessary email processing.

3. If the attachment of an email is too large to put into the queue, we can put it into the attachment store (s3).

4. Emails are put in the incoming email queue. The queue decouples mail processing workers from SMTP servers so they can be scaled independently. Moreover, the queue serves as a buffer in case the email volume surges.

5. Mail processing workers are responsible for a lot of tasks, including filtering out spam mails, stopping viruses, etc. The following steps assume an email passed the validation.

6. The email is stored in the mail storage, cache, and object data store.

7. If the receiver is currently online, the email is pushed to real-time servers.

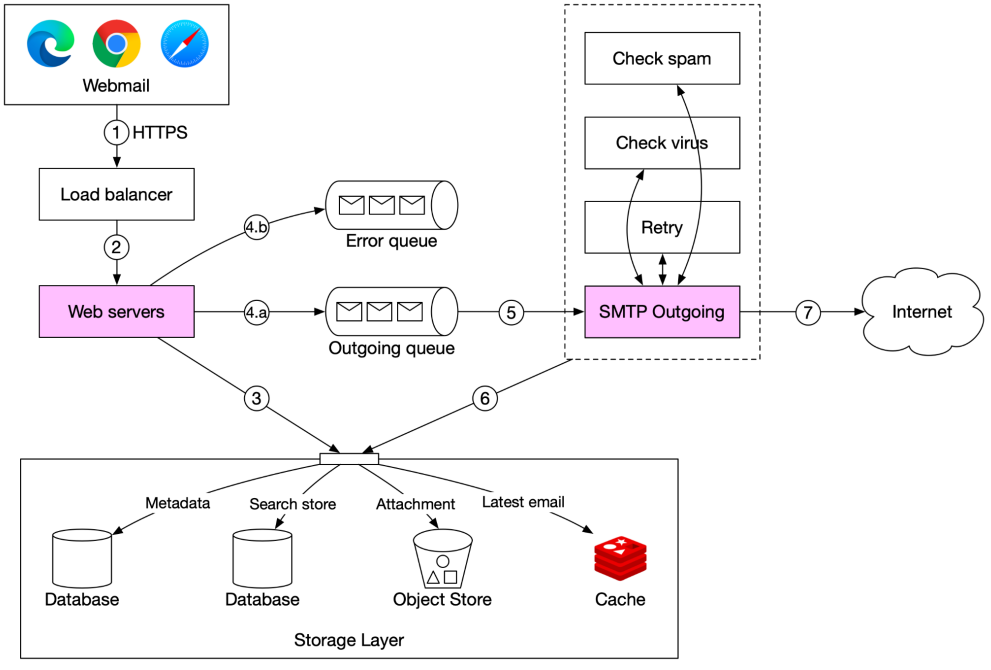
8. Real-time servers are WebSocket servers that allow clients to receive new emails in real-time.

9. For offline users, emails are stored in the storage layer. When a user comes back online, the webmail client connects to web servers via RESTful API.

10. Web servers pull new emails from the storage layer and return them to the client.

Email sending flow

In this post, we will take a closer look at the email sending flow.



1. A user writes an email on webmail and presses the “send” button. The request is sent to the load balancer.

2. The load balancer makes sure it doesn’t exceed the rate limit and routes traffic to web servers.

3. Web servers are responsible for:

- Basic email validation. Each incoming email is checked against pre-defined rules such as email size limit.

- Checking if the domain of the recipient’s email address is the same as the sender. If it is the same, email data is inserted to storage, cache, and object store directly. The recipient can fetch the email directly via the RESTful API. There is no need to go to step 4.

4. Message queues.

4.a. If basic email validation succeeds, the email data is passed to the outgoing queue.

4.b. If basic email validation fails, the email is put in the error queue.

5. SMTP outgoing workers pull events from the outgoing queue and make sure emails are spam and virus free.

6. The outgoing email is stored in the “Sent Folder” of the storage layer.

7. SMTP outgoing workers send the email to the recipient mail server.

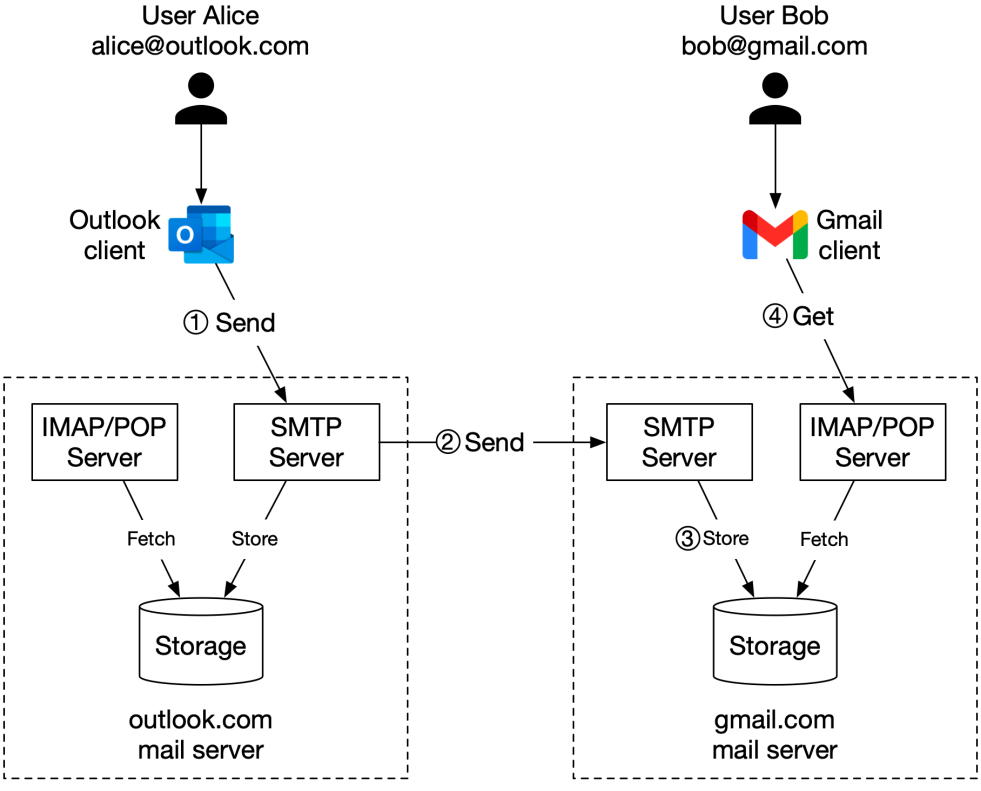
Each message in the outgoing queue contains all the metadata required to create an email. A distributed message queue is a critical component that allows asynchronous mail processing. By decoupling SMTP outgoing workers from the web servers, we can scale SMTP outgoing workers independently.

We monitor the size of the outgoing queue very closely. If there are many emails stuck in the queue, we need to analyze the cause of the issue. Here are some possibilities:
- The recipient’s mail server is unavailable. In this case, we need to retry sending the email at a later time. Exponential backoff might be a good retry strategy.

- Not enough consumers to send emails. In this case, we may need more consumers to reduce the processing time.

Interview Question: Design Gmail

One picture is worth more than a thousand words. In this post, we will take a look at what happens when Alice sends an email to Bob.



1. Alice logs in to her Outlook client, composes an email, and presses “send”. The email is sent to the Outlook mail server. The communication protocol between the Outlook client and mail server is SMTP.

2. Outlook mail server queries the DNS (not shown in the diagram) to find the address of the recipient’s SMTP server. In this case, it is Gmail’s SMTP server. Next, it transfers the email to the Gmail mail server. The communication protocol between the mail servers is SMTP.

3. The Gmail server stores the email and makes it available to Bob, the recipient.

4. Gmail client fetches new emails through the IMAP/POP server when Bob logs in to Gmail.

Please keep in mind this is a highly simplified design. Hope it sparks your interest and curiosity:) I'll explain each component in more depth in the future.

Map rendering

Google Maps Continued. Let's take a look at **Map Rendering** in this post.

Pre-Computed Tiles

One foundational concept in map rendering is tiling. Instead of rendering the entire map as one large custom image, the world is broken up into smaller tiles. The client only downloads the relevant tiles for the area the user is in and stitches them together like a mosaic for display. The tiles are pre-computed at different zoom levels. Google Maps uses 21 zoom levels.

For example, at zoom level 0, The entire map is represented by a single tile of size 256 * 256 pixels. Then at zoom level 1, the number of map tiles doubles in both north-south and east-west directions, while each tile stays at 256 * 256 pixels. So we have 4 tiles at zoom level 1, and the whole image of zoom level 1 is 512 * 512 pixels. With each increment, the entire set of tiles has 4x as many pixels as the previous level. The increased pixel count provides an increasing level of detail to the user.

This allows the client to render the map at the best granularities depending on the client's zoom level without consuming excessive bandwidth to download tiles with too much detail. This is especially important when we are loading the images from mobile clients.

Road Segments

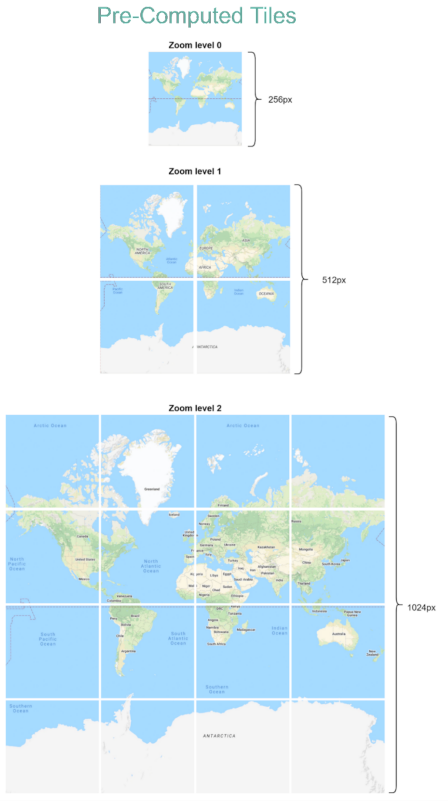
Now that we have transformed massive maps into tiles, we also need to define a data structure for the roads. We divide the world of roads into small blocks. We call these blocks road segments. Each road segment contains multiple roads, junctions, and other metadata.

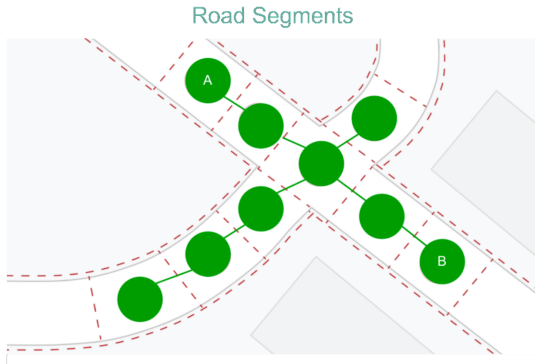
We group nearby segments into super segments. This process can be applied repeatedly to meet the level of coverage required.

We then transform the road segments into a data structure that the navigation algorithms can use. The typical approach is to convert the map into a ***graph***, where the nodes are road segments, and two nodes are connected if the corresponding road segments are reachable

neighbors. In this way, finding a path between two locations becomes a shortest-path problem, where we can leverage Dijkstra or A* algorithms.

Google Maps - Map Rendering



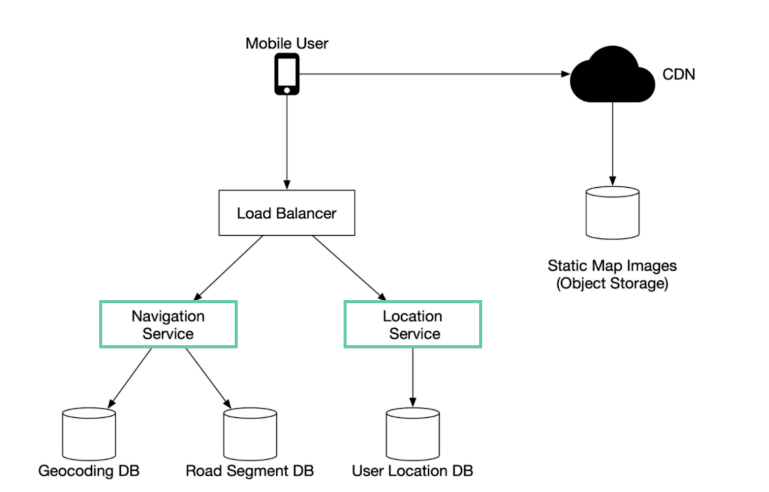


Interview Question: Design Google Maps

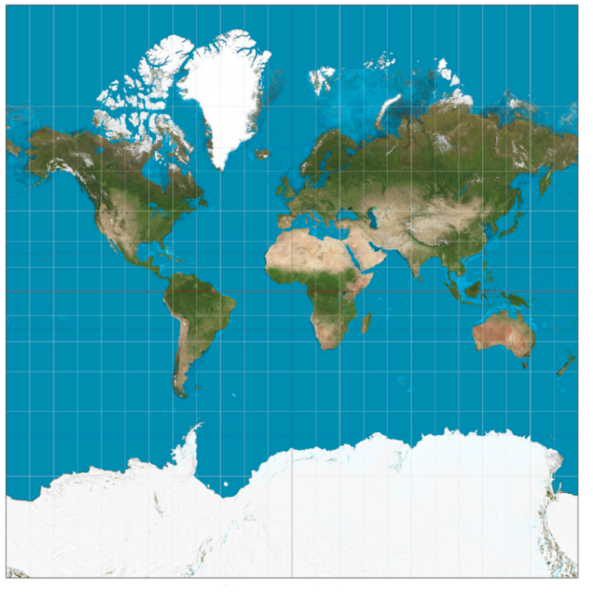
Google started project Google Maps in 2005. As of March 2021, Google Maps had one billion daily active users, 99% coverage of the world in 200 countries.

Although Google Maps is a very complex system, we can break it down into 3 high-level components. In this post, let's take a look at how to design a simplified Google Maps.

Google Maps



2D Map Projection



Location Service

The location service is responsible for recording a user's location update. The Google Map clients send location updates every few seconds. The user location data is used in many cases:

- detect new and recently closed roads
- improve the accuracy of the map over time
- used as an input for live traffic data.

Map Rendering

The world's map is projected into a huge 2D map image. It is broken down into small image blocks called "tiles" (see below). The tiles are static. They don't change very often. An efficient way to serve static tile files is with a CDN backed by cloud storage like S3. The users can load the necessary tiles to compose a map from nearby CDN.

What if a user is zooming and panning the map viewpoint on the client to explore their surroundings?

An efficient way is to pre-calculate the map blocks with different zoom levels and load the images when needed.

Navigation Service

This component is responsible for finding a reasonably fast route from point A to point B. It calls two services to help with the path calculation:

1 Geocoding Service: resolve the given address to a latitude/longitude pair

2 Route Planner Service: this service does three things in sequence:

- Calculate the top-K shortest paths between A and B
- Calculate the estimation of time for each path based on current traffic and historical data
- Rank the paths by time predictions and user filtering. For example, the user doesn't want to avoid tolls.

Pull vs push models

There are two ways metrics data can be collected, pull or push. It is a routine debate as to which one is better and there is no clear answer. In this post, we will take a look at the pull model.

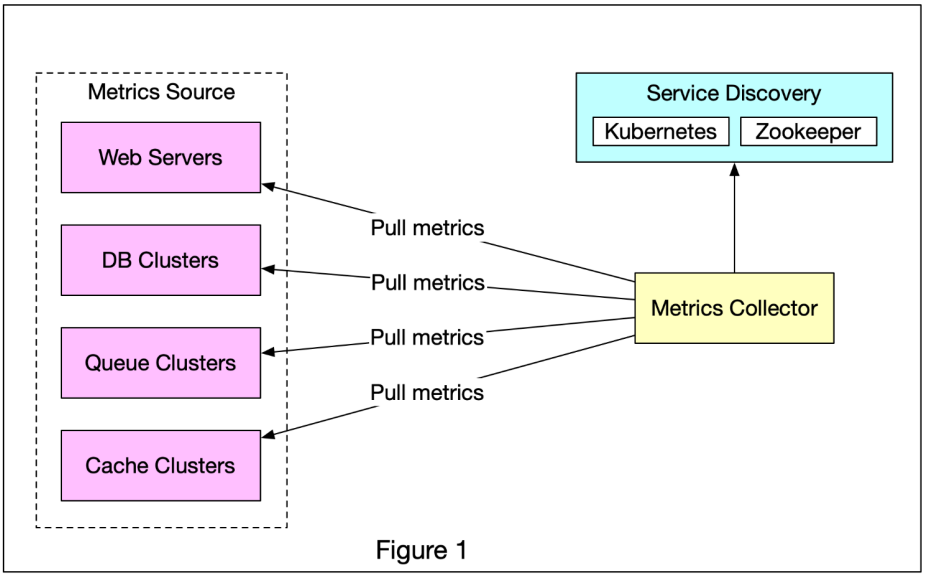


Figure 1

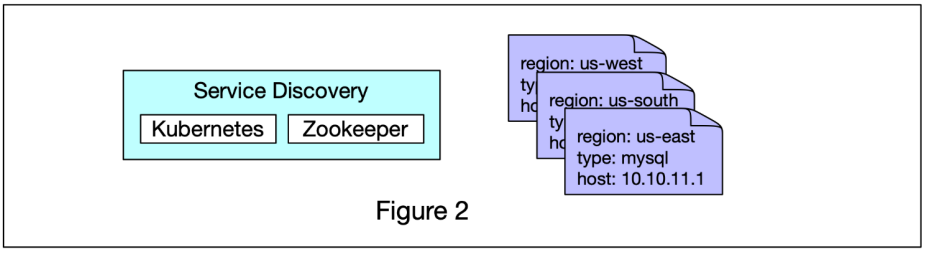


Figure 2

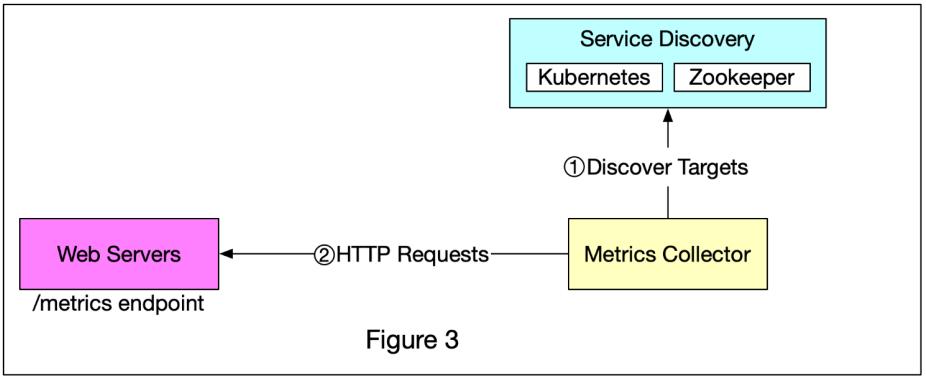


Figure 3

Figure 1 shows data collection with a pull model over HTTP. We have dedicated metric collectors which pull metrics values from the running applications periodically.

In this approach, the metrics collector needs to know the complete list of service endpoints to pull data from. One naive approach is to use a file to hold DNS/IP information for every service endpoint on the “metric collector” servers. While the idea is simple, this approach is hard to maintain in a large-scale environment where servers are added or removed frequently, and we want to ensure that metric collectors don’t miss out on collecting metrics from any new servers.

The good news is that we have a reliable, scalable, and maintainable solution available through Service Discovery, provided by Kubernetes, Zookeeper, etc., wherein services register their availability and the metrics collector can be notified by the Service Discovery component whenever the list of service endpoints changes. Service discovery contains configuration rules about when and where to collect metrics as shown in Figure 2.

Figure 3 explains the pull model in detail.

1 The metrics collector fetches configuration metadata of service endpoints from Service Discovery. Metadata include pulling interval, IP addresses, timeout and retries parameters, etc.

2 The metrics collector pulls metrics data via a pre-defined HTTP endpoint (for example, /metrics). To expose the endpoint, a client library usually needs to be added to the service. In Figure 3, the service is Web Servers.

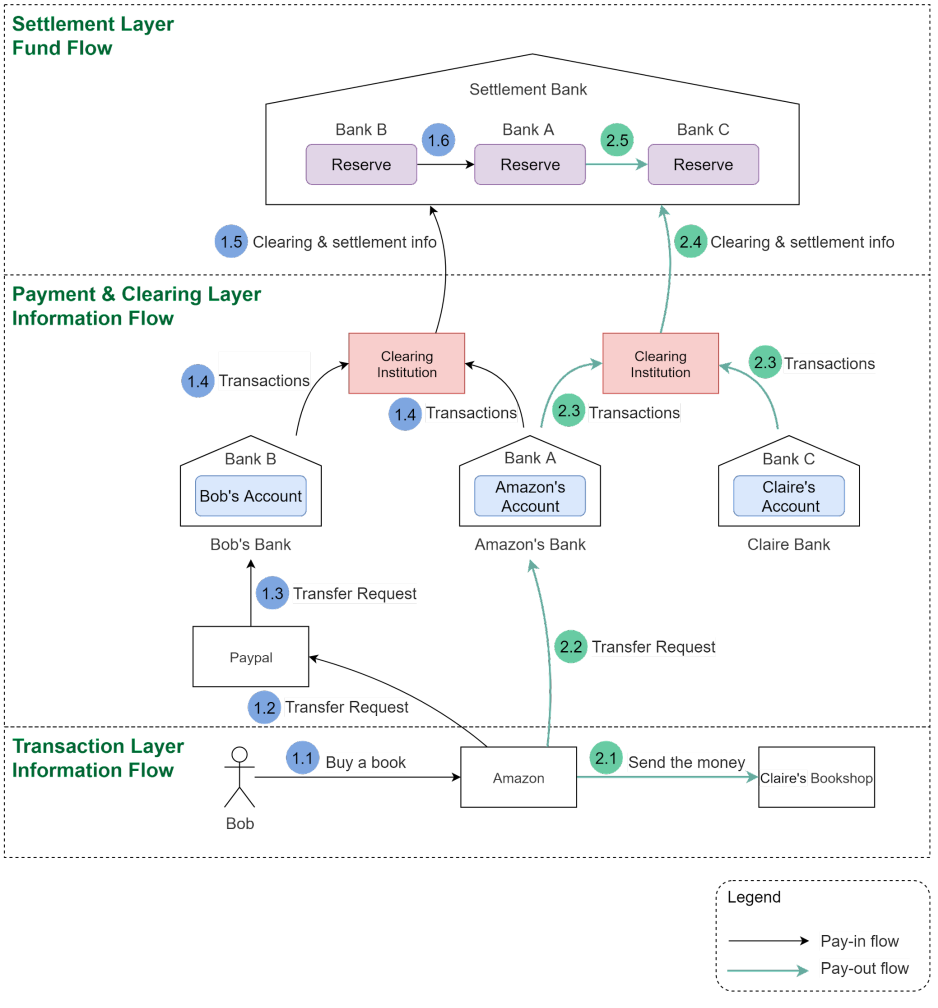
3 Optionally, the metrics collector registers a change event notification with Service Discovery to receive an update whenever the service endpoints change. Alternatively, the metrics collector can poll for endpoint changes periodically.

Money movement

One picture is worth more than a thousand words. This is what happens when you buy a product using Paypal/bank card under the hood.

To understand this, we need to digest two concepts: **clearing** & **settlement**. Clearing is a process that calculates who should pay whom with how much money; while settlement is a process where real money moves between reserves in the settlement bank.

Money Movement



Let’s say Bob wants to buy an SDI book from Claire’s shop on Amazon.

- Pay-in flow (Bob pays Amazon money):
1.1 Bob buys a book on Amazon using Paypal.
1.2 Amazon issues a money transfer request to Paypal.
1.3 Since the payment token of Bob’s debit card is stored in Paypal, Paypal can transfer money, on Bob’s behalf, to Amazon’s bank account in Bank A.
1.4 Both Bank A and Bank B send transaction statements to the clearing institution. It reduces the transactions that need to be settled. Let’s assume Bank A owns Bank B $100 and Bank B owns bank A $500 at the end of the day. When they settle, the net position is that Bank B pays Bank A $400.
1.5 & 1.6 The clearing institution sends clearing and settlement information to the settlement bank. Both Bank A and Bank B have pre-deposited funds in the settlement bank as money reserves, so actual money movement happens between two reserve accounts in the settlement bank.

- Pay-out flow (Amazon pays the money to the seller: Claire):
2.1 Amazon informs the seller (Claire) that she will get paid soon.
2.2 Amazon issues a money transfer request from its own bank (Bank A) to the seller bank (bank C). Here both banks record the transactions, but no real money is moved.
2.3 Both Bank A and Bank C send transaction statements to the clearing institution.
2.4 & 2.5 The clearing institution sends clearing and settlement information to the settlement bank. Money is transferred from Bank A’s reserve to Bank C’s reserve.

Notice that we have three layers:
- Transaction layer: where the online purchases happen
- Payment and clearing layer: where the payment instructions and transaction netting happen
- Settlement layer: where the actual money movement happen

The first two layers are called information flow, and the settlement layer is called fund flow.

You can see the **information flow and fund flow are separated**. In the info flow, the money seems to be deducted from one bank account and added to another bank account, but the actual money movement happens in the settlement bank at the end of the day.

Because of the asynchronous nature of the info flow and the fund flow, reconciliation is very important for data consistency in the systems along with the flow.

It makes things even more interesting when Bob wants to buy a book in the Indian market, where Bob pays USD but the seller can only receive INR.

Reconciliation

My previous post about painful payment reconciliation problems sparked lots of interesting discussions. One of the readers shared more problems we may face when working with intermediary payment processors in the trenches and a potential solution:

1. Foreign Currency Problem: When you operate a store globally, you will come across this problem quite frequently. To go back to the example from Paypal - if the transaction happens in a currency different from the standard currency of Paypal, this will create another layer, where the transaction is first received in that currency and exchanged to whatever currency your Paypal is using. There needs to be a reliable way to reconcile that currency exchange transaction. It certainly does not help that every payment provider handles this differently.

2. Payment providers are only that - intermediaries. Each purchase does not trigger two events for a company, but actually at least 4. The purchase via Paypal (where both the time and the currency dimension can come into play) trigger the debit/credit pair for the transaction and then, usually a few days later, another pair when the money is transferred from Paypal to a bank account (where there might be yet another FX discrepancy to reconcile if, for example, the initial purchase was in JPY, Paypal is set up in USD and your bank account is in EUR). There needs to be a way to reconcile all of these.

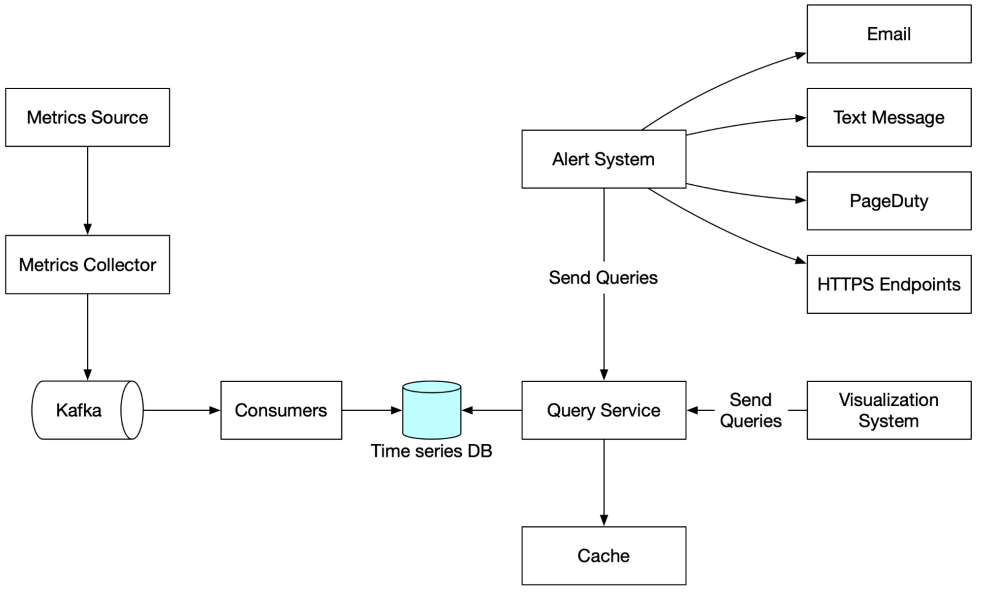
3. Some problems also pop up on the buyer side that is very platform-specific. One example is shadow transaction from Paypal: if you buy two items on Paypal with 1 week of time between the two transactions, Paypal will first debit money from your bank account for transaction A. If at the time of transaction B, transaction A has not gone through completely or is canceled, there might be a world where Paypal will use the money from transaction A to partially pay for transaction B, which leads to only a partial amount of transaction B being withdrawn from the bank account.

In practice, this usually looks something like this:
1) Your shop assigns an order number to the purchase

2) The order number is carried over to the payment provider
3) The payment provider creates another internal ID, which is carried over across transactions within the system
4) The payment ID is used when you get the payout on your bank account (or the payment provider bundles individual payments, which can be reconciled within the payment provider system)
5) Ideally, your payment provider and your shop have an integration/API with the tool you use to (hopefully automatically) create invoices. This usually carries over the order id from the shop (closing the loop) and sometimes even the payment id to match it with the invoice id, which you then can use to reconcile it with your accounts receivable/payable. :)

Credit: A knowledgeable reader who prefers to stay private. Thank you!

Continued: how to choose the right database for metrics collecting service?



There are many storage systems available that are optimized for time-series data. The optimization lets us use far fewer servers to handle the same volume of data. Many of these databases also have custom query interfaces specially designed for the analysis of time-series data that are much easier to use than SQL. Some even provide features to manage data retention and data aggregation. Here are a few examples of time-series databases.

OpenTSDB is a distributed time-series database, but since it is based on Hadoop and HBase, running a Hadoop/HBase cluster adds complexity. Twitter uses MetricsDB, and Amazon offers Timestream as a time-series database. According to DB-engines, the two most popular time-series databases are InfluxDB and Prometheus, which are designed to store large volumes of time-series data and quickly perform real-time analysis on that data. Both of them primarily rely on an in-memory cache and on-disk storage. And they both handle durability and performance quite well. According to the benchmark, an InfluxDB with 8 cores and 32GB RAM can handle over 250,000 writes per second.

Since a time-series database is a specialized database, you are not expected to understand the internals in an interview unless you explicitly mentioned it in your resume. For the purpose of an interview, it's important to understand the metrics data are time-series in nature and we can select time-series databases such as InfluxDB for storage to store them.

Another feature of a strong time-series database is efficient aggregation and analysis of a large amount of time-series data by labels, also known as tags in some databases. For example, InfluxDB builds indexes on labels to facilitate the fast lookup of time-series by labels. It provides clear best-practice guidelines on how to use labels, without overloading the database. The key is to make sure each label is of low cardinality (having a small set of possible values). This feature is critical for visualization, and it would take a lot of effort to build this with a general-purpose database.

Which database shall I use for the metrics collecting system?

This is one of the most important questions we need to address in an interview.

Data access pattern
As shown in the diagram, each label on the y-axis represents a time series (uniquely identified by the names and labels) while the x-axis represents time.

The write load is heavy. As you can see, there can be many time-series data points written at any moment. There are millions of operational metrics written per day, and many metrics are collected at high frequency, so the traffic is undoubtedly write-heavy.

At the same time, the read load is spiky. Both visualization and alert services send queries to the database and depending on the access patterns of the graphs and alerts, the read volume could be bursty.

Choose the right database
The data storage system is the heart of the design. It's not recommended to build your own storage system or use a general-purpose storage system (MySQL) for this job.

A general-purpose database, in theory, could support time-series data, but it would require expert-level tuning to make it work at our scale. Specifically, a relational database is not optimized for operations you would commonly perform against time-series data. For example, computing the moving average in a rolling time window requires complicated SQL that is difficult to read (there is an example of this in the deep dive section). Besides, to support tagging/labeling data, we need to add an index for each tag. Moreover, a general-purpose relational database does not perform well under constant heavy write load. At our scale, we would need to expend significant effort in tuning the database, and even then, it might not perform well.

How about NoSQL? In theory, a few NoSQL databases on the market could handle time-series data effectively. For example, Cassandra and Bigtable can both be used for time series data. However, this would require deep knowledge of the internal workings of each NoSQL to devise a scalable schema for effectively storing and querying time-series data. With industrial-scale time-series databases readily available, using a general purpose NoSQL database is not appealing.

There are many storage systems available that are optimized for time-series data. The optimization lets us use far fewer servers to handle the same volume of data. Many of these databases also have custom query interfaces specially designed for the analysis of time-series data that are much easier to use than SQL. Some even provide features to manage data retention and data aggregation. Here are a few examples of time-series databases.

OpenTSDB is a distributed time-series database, but since it is based on Hadoop and HBase, running a Hadoop/HBase cluster adds complexity. Twitter uses MetricsDB, and Amazon offers Timestream as a time-series database. According to DB-engines, the two most popular time-series databases are InfluxDB and Prometheus, which are designed to store large volumes of time-series data and quickly perform real-time analysis on that data. Both of them primarily rely on an in-memory cache and on-disk storage. And they both handle durability and performance quite well. According to the benchmark listed on InfluxDB website, a DB server with 8 cores and 32GB RAM can handle over 250,000 writes per second.

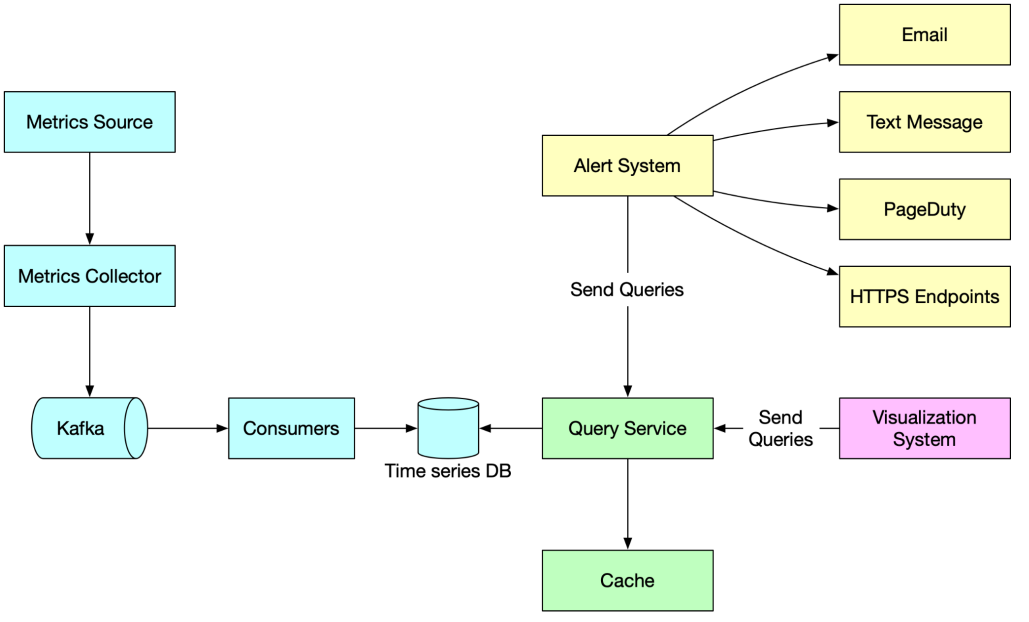
Since a time-series database is a specialized database, you are not expected to understand the internals in an interview unless you explicitly mentioned it in your resume. For the purpose of an interview, it’s important to understand the metrics data are time-series in nature and we can select time-series databases such as InfluxDB for storage to store them.

Another feature of a strong time-series database is efficient aggregation and analysis of a large amount of time-series data by labels, also known as tags in some databases. For example, InfluxDB builds indexes on labels to facilitate the fast lookup of time-series by

labels. It provides clear best-practice guidelines on how to use labels, without overloading the database. The key is to make sure each label is of low cardinality (having a small set of possible values). This feature is critical for visualization, and it would take a lot of effort to build this with a general-purpose database.

Metrics monitoring and altering system

A well-designed **metrics monitoring** and alerting system plays a key role in providing clear visibility into the health of the infrastructure to ensure high availability and reliability. The diagram below explains how it works at a high level.



Metrics source: This can be application servers, SQL databases, message queues, etc.

Metrics collector: It gathers metrics data and writes data into the time-series database.

Time-series database: This stores metrics data as time series. It usually provides a custom query interface for analyzing and summarizing a large amount of time-series data. It maintains indexes on labels to facilitate the fast lookup of time-series data by labels.

Kafka: Kafka is used as a highly reliable and scalable distributed messaging platform. It decouples the data collection and data processing services from each other.

Consumers: Consumers or streaming processing services such as Apache Storm, Flink and Spark, process and push data to the time-series database.

Query service: The query service makes it easy to query and retrieve data from the time-series database. This should be a very thin wrapper if we choose a good time-series database. It could also be entirely replaced by the time-series database's own query interface.

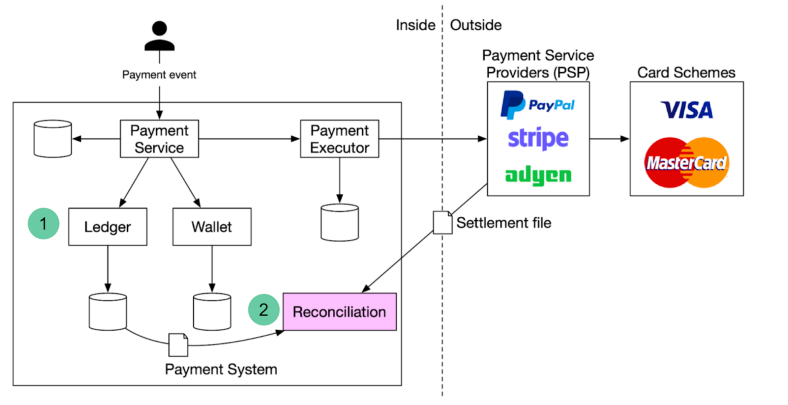
Alerting system: This sends alert notifications to various alerting destinations.

Visualization system: This shows metrics in the form of various graphs/charts.

Reconciliation

Reconciliation might be the most painful process in a payment system. It is the process of comparing records in different systems to make sure the amounts match each other.

Reconciliation in Payment System



Double-entry Bookkeeping in Ledger

Account	Debit	Credit
buyer	$200	
seller		$200

For example, if you pay $200 to buy a watch with Paypal:
- The eCommerce website should have a record about the purchase order of $200.
- There should be a transaction record of $200 in Paypal (marked with 2 in the diagram).
- The Ledger should record a debit of $200 dollars for the buyer, and a credit of $200 for the seller. This is called double-entry bookkeeping (see the table below).

Let's take a look at some pain points and how we can address them:

Problem 1: Data normalization. When comparing records in different systems, they come in different formats. For example, the timestamp can be “2022/01/01” in one system and “Jan 1, 2022” in another.
Possible solution: we can add a layer to transform different formats into the same format.

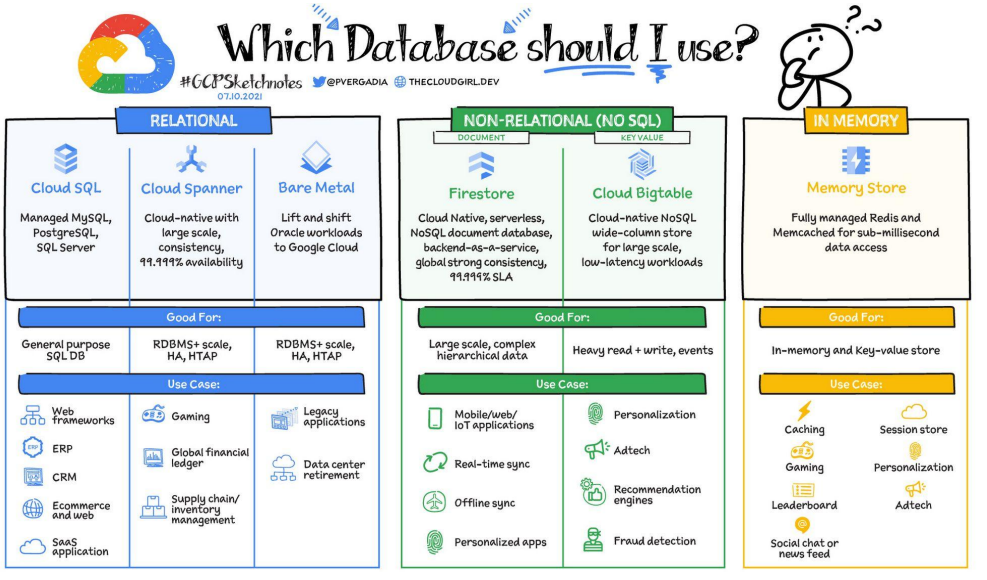
Problem 2: Massive data volume
Possible solution: we can use big data processing techniques to speed up data comparisons. If we need near real-time reconciliation, a streaming platform such as Flink is used; otherwise, end-of-day batch processing such as Hadoop is enough.

Problem 3: Cut-off time issue. For example, if we choose 00:00:00 as the daily cut-off time, one record is stamped with 23:59:55 in the internal system, but might be stamped 00:00:30 in the external system (Paypal), which is the next day. In this case, we couldn’t find this record in today’s Paypal records. It causes a discrepancy.
Possible solution: we need to categorize this break as a “temporary break” and run it later against the next day’s Paypal records. If we find a match in the next day’s Paypal records, the break is cleared, and no more action is needed.

You may argue that if we have exactly-once semantics in the system, there shouldn’t be any discrepancies. But the truth is, there are so many places that can go wrong. Having a reconciliation system is always necessary. It is like having a safety net to keep you sleeping well at night.

Which database shall I use? This is one of the most important questions we usually need to address in an interview.

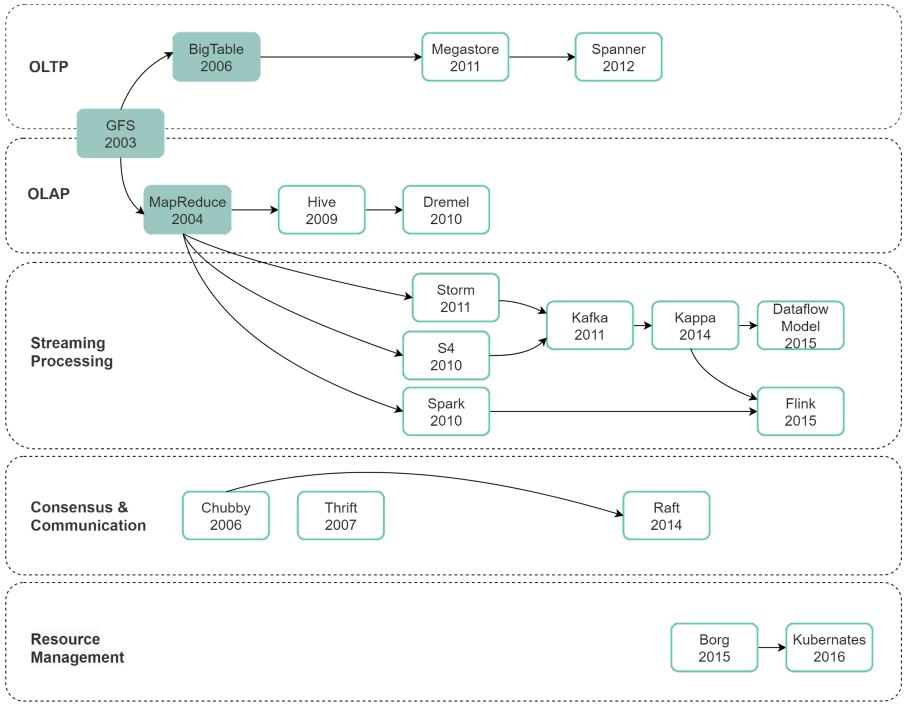
Choosing the right database is hard. Google Cloud recently posted a great article that summarized different database options available in Google Cloud and explained which use cases are best suited for each database option.



Big data papers

Below is a timeline of important big data papers and how the techniques evolved over time.

Big Data Theses Timeline & Relationship



The green highlighted boxes are the famous 3 Google papers, which established the foundation of the big data framework. At the high-level:

Big Data Techniques = *Massive data* + *Massive calculation*

Let's look at the **OLTP** evolution. BigTable provided a distributed storage system for structured data but dropped some characteristics of relational DB. Then Megastore brought back schema and simple transactions; Spanner brought back data consistency.

Now let's look at the **OLAP** evolution. MapReduce was not easy to program, so Hive solved this by introducing a SQL-like query

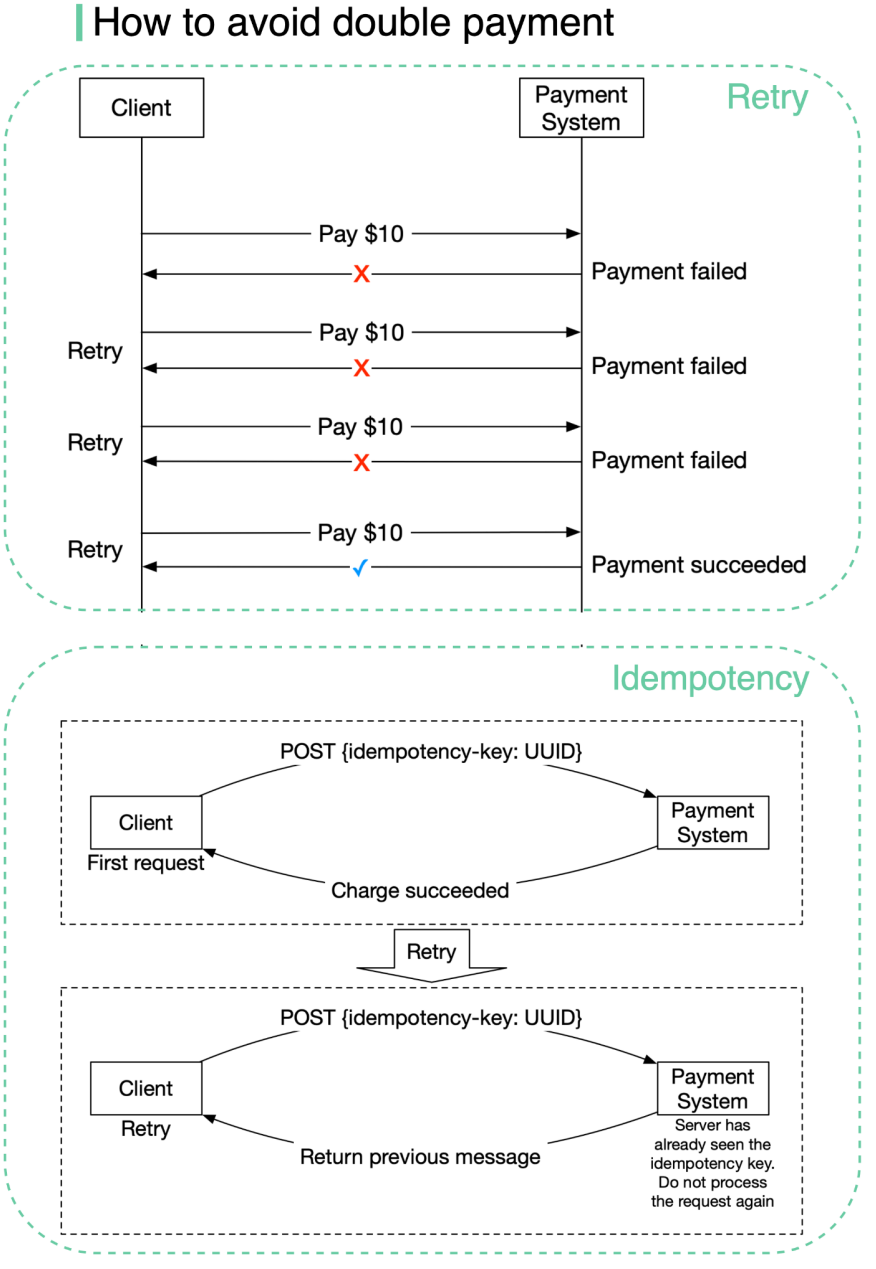
language. But Hive still used MapReduce under the hood, so it's not very responsive. In 2010, Dremel provided an interactive query engine.

Streaming processing was born to further solve the latency issue in OLAP. The famous ***lambda*** architecture was based on Storm and MapReduce, where streaming processing and batch processing have different processing flows. Then people started to build streaming processing with apache Kafka. ***Kappa*** architecture was proposed in 2014, where streaming and batching processings were merged into one flow. Google published The Dataflow Model in 2015, which was an abstraction standard for streaming processing, and Flink implemented this model.

To manage a big crowd of commodity server resources, we need resource management Kubernetes.

Avoid double charge

One of the most serious problems a payment system can have is to **double charge a customer**. When we design the payment system, it is important to guarantee that the payment system executes a payment order exactly-once.



At the first glance, exactly-once delivery seems very hard to tackle, but if we divide the problem into two parts, it is much easier to solve. Mathematically, an operation is executed exactly-once if:

1. It is executed at least once.

2. At the same time, it is executed at most once.

We now explain how to implement at least once using retry and at most once using idempotency check.

Retry

Occasionally, we need to retry a payment transaction due to network errors or timeout. Retry provides the at-least-once guarantee. For example, as shown in Figure 10, the client tries to make a $10 payment, but the payment keeps failing due to a poor network connection. Considering the network condition might get better, the client retries the request and this payment finally succeeds at the fourth attempt.

Idempotency

From an API standpoint, idempotency means clients can make the same call repeatedly and produce the same result.

For communication between clients (web and mobile applications) and servers, an idempotency key is usually a unique value that is generated by clients and expires after a certain period of time. A UUID is commonly used as an idempotency key and it is recommended by many tech companies such as Stripe and PayPal. To perform an idempotent payment request, an idempotency key is added to the HTTP header: *<idempotency-key: key_value>*.

Payment security

A few weeks ago, I posted the high-level design for the payment system. Today, I'll continue the discussion and focus on payment security.

The table below summarizes techniques that are commonly used in payment security. If you have any questions or I missed anything, please leave a comment.

Problem	Solution
Request/response eavesdropping	Use HTTPS
Data tampering	Enforce encryption and integrity monitoring
Man-in-the-middle attack	Use SSL and authentication certificates
Data loss	Database replication across multiple regions and take snapshot of data
Distributed denial-of-service attack (DDoS)	Rate limiting and firewall
Card theft	Tokenization. Instead of using real card numbers, tokens are stored and used for payment
PCI compliance	PCI DSS is an information security standard for organizations that handle branded credit cards
Fraud	Address verification, card verification value (CVV), user behavior analysis, etc.

System Design Interview Tip

One pro tip for acing a system design interview is to read the engineering blog of the company you are interviewing with. You can get a good sense of what technology they use, why the technology was chosen over others, and learn what issues are important to engineers.

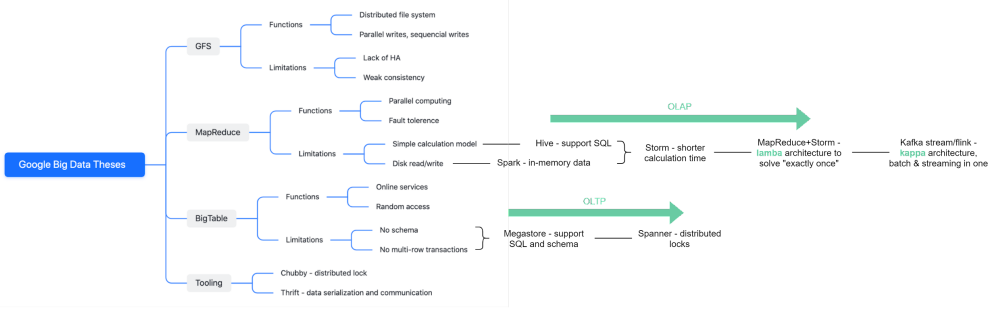


For example, here are 4 blog posts Twitter Engineering recommends:
1. The Infrastructure Behind Twitter: Scale
2. Discovery and Consumption of Analytics Data at Twitter
3. The what and why of product experimentation at Twitter
4. Twitter experimentation: technical overview

Big data evolvement

I hope everyone has a great time with friends and family during the holidays. If you are looking for some readings, classic engineering papers are a good start.

Big Data Evolvement



A lot of times when we are busy with work, we only focus on scattered information, telling us “**how**” and “what” to get our immediate needs to get things done.

However, reading the classics helps us know “**why**” behind the scenes, and teaches us how to solve problems, make better decisions, or even contribute to open source projects.

Let’s take **big data** as an example.

Big data area has progressed a lot over the past 20 years. It started from 3 Google papers (see the links in the comment), which tackled real engineering challenges at Google scale:

- GFS (2003) - big data storage
- MapReduce (2004) - calculation model
- BigTable (2006) - online services

The diagram below shows the functionalities and limitations of the 3 techniques, and how they evolve over time into two streams: OLTP and OLAP. Each evolved product was trying to solve the limitations of the

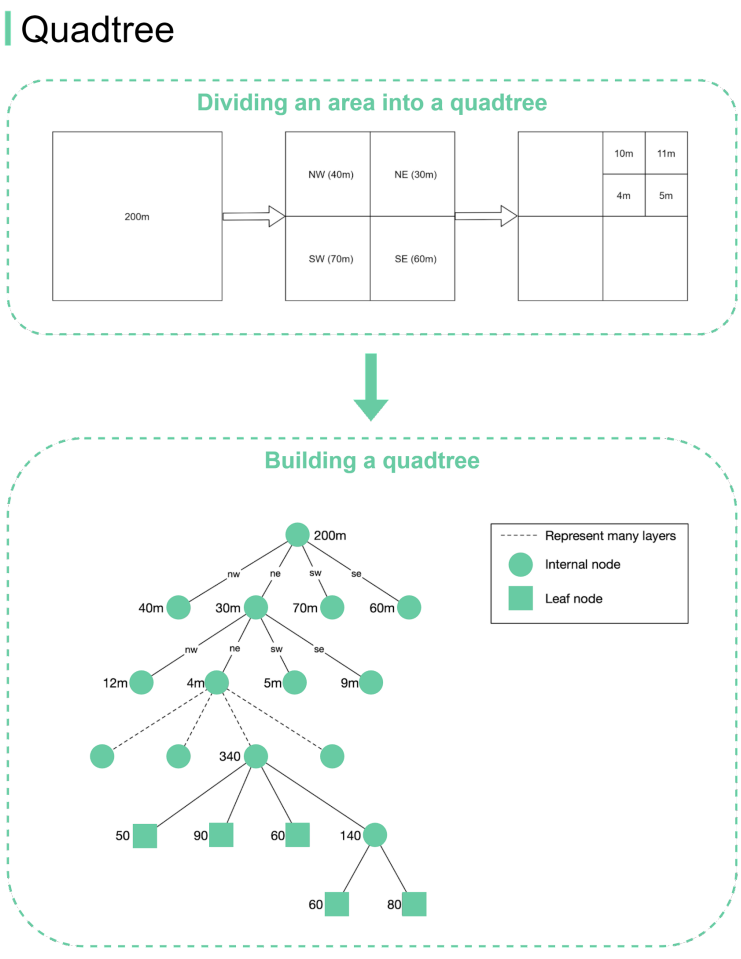
last generation. For example, “Hive - support SQL” means Hive was trying to solve the lack of SQL in MapReduce.

If you want to learn more, you can refer to the papers for details. What other classics would you recommend?

Quadtree

In this post, let's explore another data structure to find nearby restaurants on Yelp or Google Maps.

A quadtree is a data structure that is commonly used to partition a two-dimensional space by recursively subdividing it into four quadrants (grids) until the contents of the grids meet certain criteria (see the first diagram).



Quadtree is an **in-memory data structure** and it is not a database solution. It runs on each LBS (Location-Based Service, see last week's post) server, and the data structure is built at server start-up time.

The second diagram explains the quadtree building process in more detail. The root node represents the whole world map. The root node is **recursively** broken down into 4 quadrants until no nodes are left with more than 100 businesses.

How to get nearby businesses with quadtree?
- Build the quadtree in memory.

- After the quadtree is built, start searching from the root and traverse the tree, until we find the leaf node where the search origin is.

- If that leaf node has 100 businesses, return the node. Otherwise, add businesses from its neighbors until enough businesses are returned.

Update LBS server and rebuild quadtree
- It may take a few minutes to build a quadtree in memory with 200 million businesses at the server start-up time.

- While the quadtree is being built, the server cannot serve traffic.

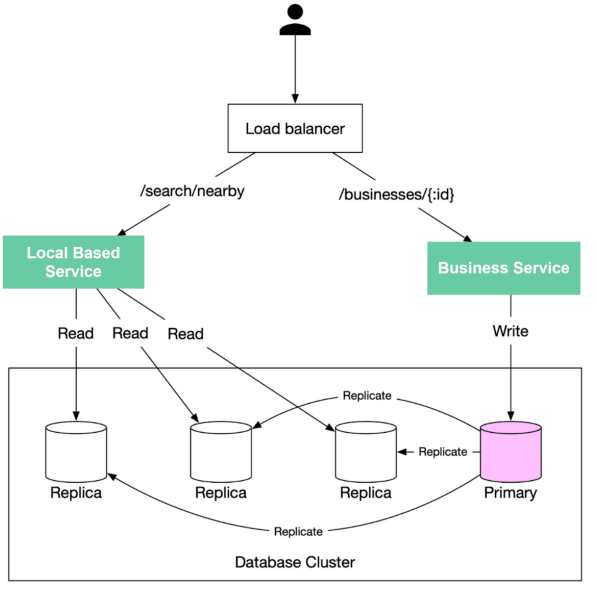
- Therefore, we should roll out a new release of the server incrementally to **a small subset** of servers at a time. This avoids taking a large swathe of the server cluster offline and causes service brownout.

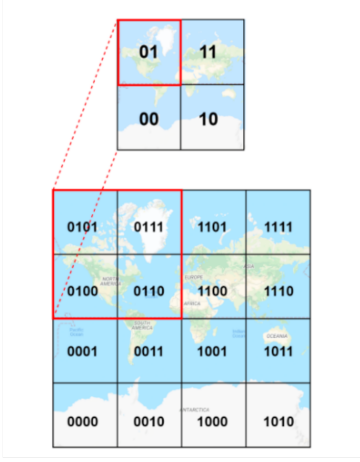
How do we find nearby restaurants on Yelp?

Here are some design details behind the scenes.

There are two key services (see the diagram below):

Proximity Service Design





- Business Service

- Add/delete/update restaurant information
- Customers view restaurant details

- Local-based Service (LBS)

- Given a radius and location, return a list of nearby restaurants

How are the restaurant locations stored in the database so that LBS can return nearby restaurants efficiently?

Store the latitude and longitude of restaurants in the database? The query will be very inefficient when you need to calculate the distance between you and every restaurant.

One way to speed up the search is using the **geohash algorithm**.

First, divide the planet into four quadrants along with the prime meridian and equator：

- Latitude range [-90, 0] is represented by 0

- Latitude range [0, 90] is represented by 1

- Longitude range [-180, 0] is represented by 0

- Longitude range [0, 180] is represented by 1

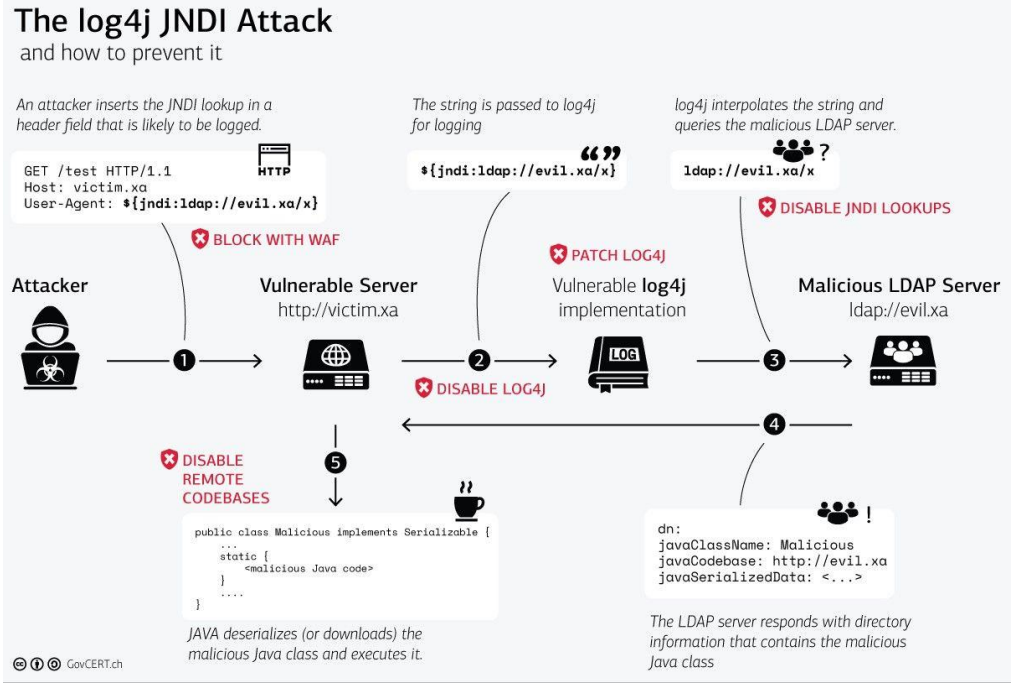
Second, divide each grid into four smaller grids. Each grid can be represented by alternating between longitude bit and latitude bit.

So when you want to search for the nearby restaurants in the red-highlighted block, you can write SQL like:

```
SELECT * FROM geohash_index WHERE geohash LIKE `01%`
```

Geohash has some limitations. There can be a lot of restaurants in one block (downtown New York), but none in another block (ocean). So there are other more complicated algorithms to optimize the process. Let me know if you are interested in the details.

One picture is worth more than a thousand words. Log4j from attack to prevention in one illustration.



Credit GovCERT

Link:
https://www.govcert.ch/blog/zero-day-exploit-targeting-popular-java-library-log4j/

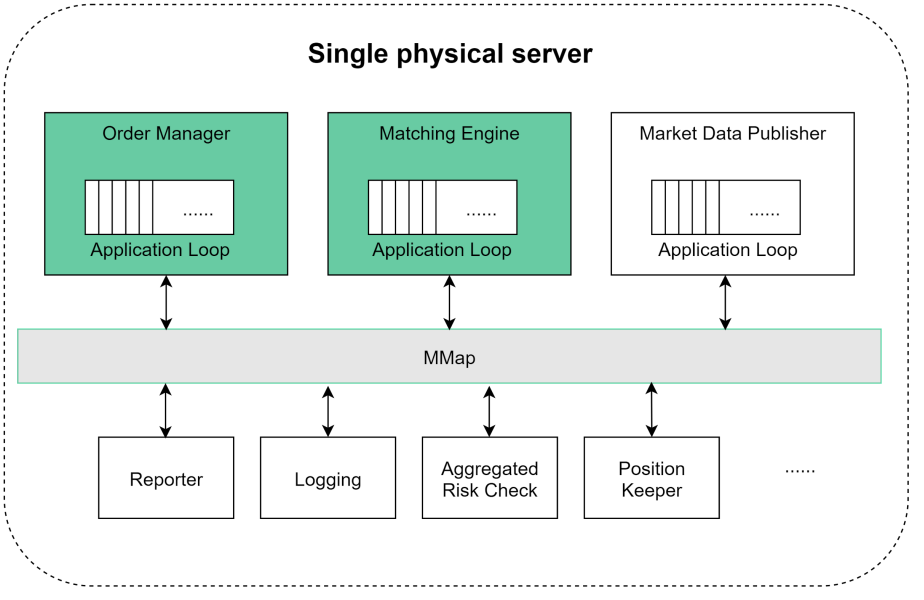
How does a modern stock exchange achieve microsecond latency?

The principal is:

Do less on the critical path !

- Fewer tasks on the critical path
- Less time on each task
- Fewer network hops
- Less disk usage

Low Latency Stock Exchange Design



For the stock exchange, the critical path is:
- **start**: an order comes into the order manager
- mandatory risk checks
- the order gets matched and the execution is sent back
- **end**: the execution comes out of the order manager

Other non-critical tasks should be removed from the critical path.

We put together a design as shown in the diagram:

- deploy all the components in a single giant server (no containers)

- use shared memory as an event bus to communicate among the components, no hard disk

- key components like Order Manager and Matching Engine are single-threaded on the critical path, and each pinned to a CPU so that there is **no context switch** and **no locks**

- the single-threaded application loop executes tasks one by one in sequence

- other components listen on the event bus and react accordingly

Match buy and sell orders

Stocks go up and down. Do you know what data structure is used to efficiently match buy and sell orders?



Buy 2700 shares: 2700 - 200 - 400 - 1100 - 100 - 900 = 0

Stock exchanges use the data structure called **order books**. An order book is an electronic list of buy and sell orders, organized by price levels. It has a buy book and a sell book, where each side of the book contains a bunch of price levels, and each price level contains a list of orders (first in first out).

The image is an example of price levels and the queued quantity in each price level.

So what happens when you place a market order to buy 2700 shares in the diagram?

- The buy order is matched with all the sell onrders at price 100.10, and the first order at price 100.11 (illustrated in light red).

- Now because of the big buy order which “eats up” the first price level on the sell book, the best ask price goes up from 100.10 to 100.11.

- So when the market is bullish, people tend to buy stocks, and the price goes up and up.

An efficient data structure for an order book must satisfy:

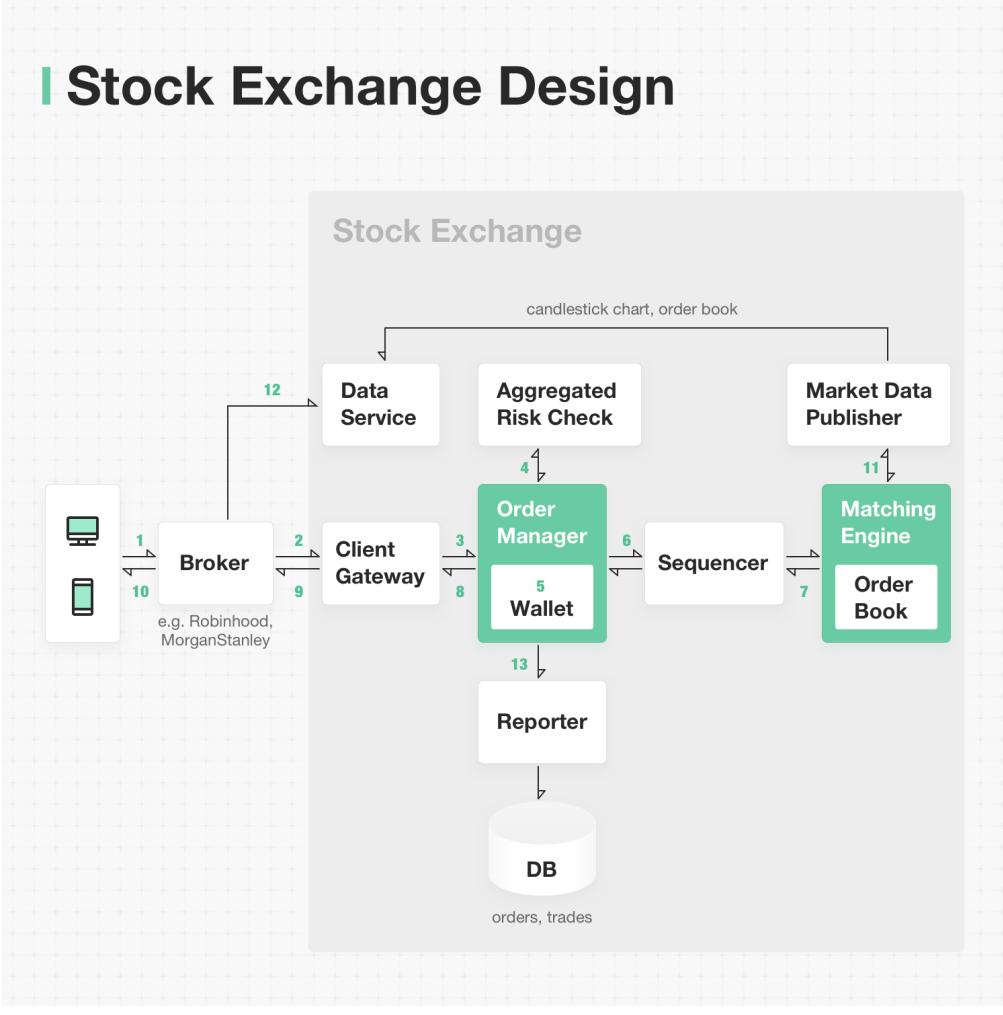
- Constant lookup time. Operations include: get volume at a price level or between price levels, query best bid/ask.

- Fast add/cancel/execute/update operations, preferably O(1) time complexity. Operations include: place a new order, cancel an order, and match an order.

Stock exchange design

The stock market has been volatile recently.

Coincidentally, we just finished a new chapter “Design a stock exchange”. I’ll use plain English to explain what happens when you place a stock buying order. The focus is on the exchange side.



Step 1: client places an order via the broker’s web or mobile app.

Step 2: broker sends the order to the exchange.

Step 3: the exchange client gateway performs operations such as validation, rate limiting, authentication, normalization, etc, and sends the order to the order manager.

Step 4: the order manager performs risk checks based on rules set by the risk manager.

Step 5: once risk checks pass, the order manager checks if there is enough balance in the wallet.

Step 6-7: the order is sent to the matching engine. The matching engine sends back the execution result if a match is found. Both order and execution results need to be sequenced first in the sequencer so that matching determinism is guaranteed.

Step 8 - 10: execution result is passed all the way back to the client.

Step 11-12: market data (including the candlestick chart and order book) are sent to the data service for consolidation. Brokers query the data service to get the market data.

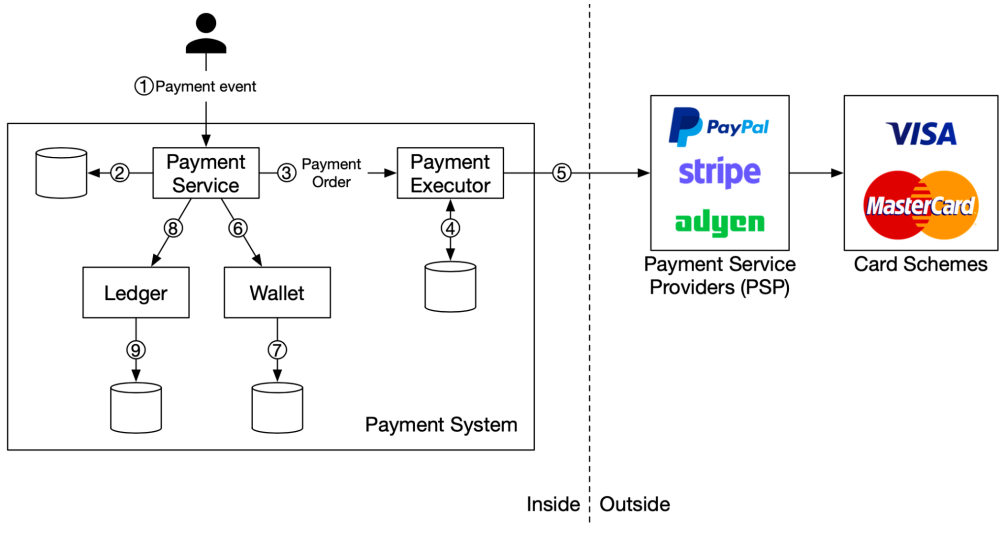
Step 13: the reporter composes all the necessary reporting fields (e.g. client_id, price, quantity, order_type, filled_quantity, remaining_quantity) and writes the data to the database for persistence

A stock exchange requires **extremely low latency**. While most web applications are ok with hundreds of milliseconds latency, a stock exchange requires **micro-second level latency**. I'll leave the latency discussion for a separate post since the post is already long.

Design a payment system

Today is Cyber Monday. Here is how money moves when you click the Buy button on Amazon or any of your favorite shopping websites.

I posted the same diagram last week for an overview and a few people asked me about the detailed steps, so here you go:



1. When a user clicks the “Buy” button, a payment event is generated and sent to the payment service.

2. The payment service stores the payment event in the database.

3. Sometimes a single payment event may contain several payment orders. For example, you may select products from multiple sellers in a single checkout process. The payment service will call the payment executor for each payment order.

4. The payment executor stores the payment order in the database.

5. The payment executor calls an external PSP to finish the credit card payment.

6. After the payment executor has successfully executed the payment, the payment service will update the wallet to record how much money a given seller has.

7. The wallet server stores the updated balance information in the database.

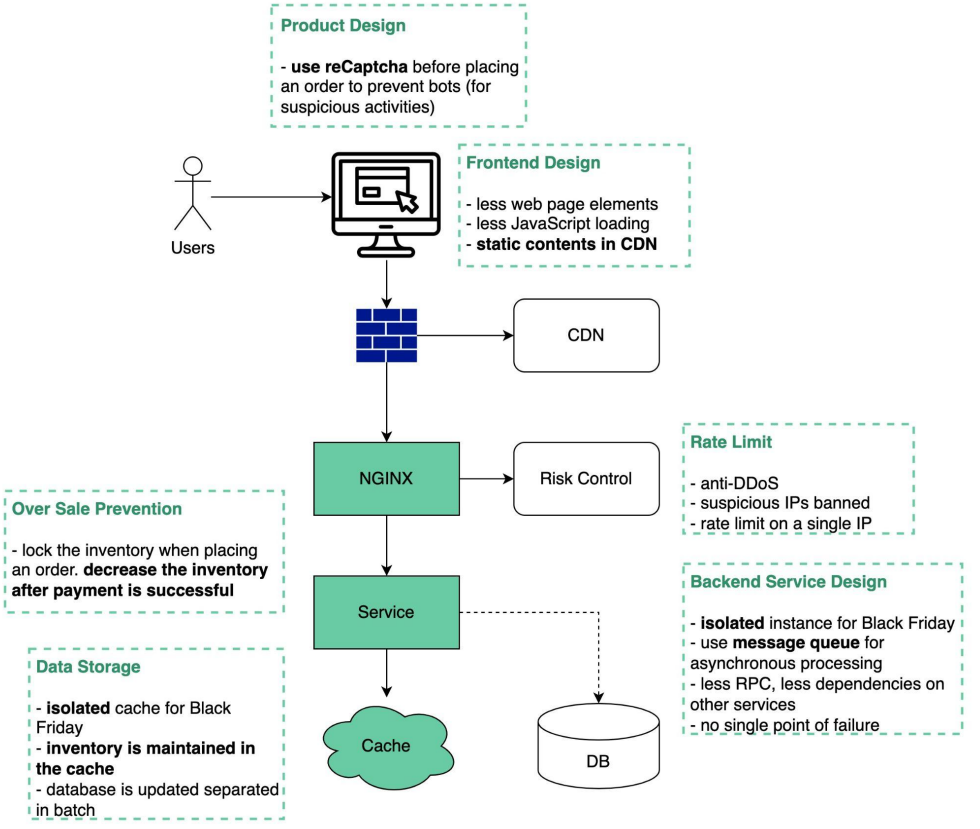
8. After the wallet service has successfully updated the seller's balance information, the payment service will call the ledger to update it.

9. The ledger service appends the new ledger information to the database.

10. Every night the PSP or banks send settlement files to their clients. The settlement file contains the balance of the bank account, together with all the transactions that took place on this bank account during the day.

Design a flash sale system

Black Friday is coming. Designing a system with extremely high concurrency, high availability and quick responsiveness needs to consider many aspects **all the way from frontend to backend.** See the below picture for details:



Design principles:

1. Less is more - less element on the web page, fewer data queries to the database, fewer web requests, fewer system dependencies
2. Short critical path - fewer hops among services or merge into one service
3. Async - use message queues to handle high TPS
4. Isolation - isolate static and dynamic contents, isolate processes and databases for rare items
5. Overselling is bad. When Decreasing the inventory is important

6. User experience is important. We definitely don’t want to inform users that they have successfully placed orders but later tell them no items are actually available

Back-of-the-envelope estimation

Recently, a few engineers asked me whether we really need back-of-the-envelope estimation in a system design interview. I think it would be helpful to clarify.

Estimations are important because we need them to understand the scale of the system and justify the design. It helps answer questions like:

- Do we really need a distributed solution?
- Is a cache layer necessary?
- Shall we choose data replication or sharding?

Here is an example of how the estimations shape the design decision.

One interview question is to design proximity service and how to scale geospatial index is a key part of it. Here are a few paragraphs we wrote to show why jumping to a sharding design without estimations is a bad idea:

“One common mistake about scaling the geospatial index is to quickly jump to a sharding scheme without considering the actual data size of the table. In our case, the full dataset for the geospatial index table is not large (quadtree index only takes 1.71G memory and storage requirement for geohash index is similar). The whole geospatial index can easily fit in the working set of a modern database server. However, depending on the read volume, a single database server might not have enough CPU or network bandwidth to service all read requests. If that is the case, it will be necessary to spread the read load among multiple database servers.

There are two general approaches to spread the load of a relational database server. We can add read replicas or shard the database.

Many engineers like to talk about sharding during interviews. However, it might not be a good fit for the geohash table. Sharding is complicated. The sharding logic has to be added to the application layer. Sometimes, sharding is the only option. In this case though, since everything can fit in the working set of a database server, there is no strong technical reason to shard the data among multiple servers.

A better approach, in this case, is to have a series of read replicas to help with the read load. This method is much simpler to develop and maintain. Thus, we recommend scaling the geospatial index table through replicas.”

—

Check out our bestselling system design books.
Paperback: Amazon Digital: ByteByteGo.